

CORUMA FOOD

MesaPronta v2.1 – JDK 21

Codigo Fonte Completo

34 arquivos

Records, Pattern Matching, Text Blocks, var, Formatted Strings

JavaFX 21.0.8 + Hibernate 6.6 + SQLite

Corumba Sistemas – 27/05/2026

INDICE

- [dependency-reduced-pom.xml](#) (2901B)
- [pom.xml](#) (4288B)
- [relatorio_testes.txt](#) (2108B)
- [src/main/java/com/corumbasistemas/corubafood/CorubaFoodApp.java](#) (4006B)
- [src/main/java/com/corumbasistemas/corubafood/controller/AuthController.java](#) (1611B)
- [src/main/java/com/corumbasistemas/corubafood/controller/ConfigIntegracaoController.java](#) (10394B)
- [src/main/java/com/corumbasistemas/corubafood/controller/DeliveryController.java](#) (17574B)
- [src/main/java/com/corumbasistemas/corubafood/controller/LoginController.java](#) (1786B)
- [src/main/java/com/corumbasistemas/corubafood/controller/PagamentoController.java](#) (1225B)
- [src/main/java/com/corumbasistemas/corubafood/controller/PrincipalController.java](#) (7585B)
- [src/main/java/com/corumbasistemas/corubafood/model/Categoria.java](#) (1606B)
- [src/main/java/com/corumbasistemas/corubafood/model/Empresa.java](#) (1852B)
- [src/main/java/com/corumbasistemas/corubafood/model/ItemPedido.java](#) (1732B)
- [src/main/java/com/corumbasistemas/corubafood/model/ItemPedidoDelivery.java](#) (3118B)
- [src/main/java/com/corumbasistemas/corubafood/model/Mesa.java](#) (3490B)
- [src/main/java/com/corumbasistemas/corubafood/model/Pagamento.java](#) (2081B)
- [src/main/java/com/corumbasistemas/corubafood/model/Pedido.java](#) (2665B)
- [src/main/java/com/corumbasistemas/corubafood/model/PedidoDelivery.java](#) (6998B)
- [src/main/java/com/corumbasistemas/corubafood/model/Produto.java](#) (2457B)
- [src/main/java/com/corumbasistemas/corubafood/model/Usuario.java](#) (1875B)
- [src/main/java/com/corumbasistemas/corubafood/repository/PagamentoRepository.java](#) (1227B)
- [src/main/java/com/corumbasistemas/corubafood/repository/ProdutoRepository.java](#) (1526B)
- [src/main/java/com/corumbasistemas/corubafood/repository/UsuarioRepository.java](#) (1226B)
- [src/main/java/com/corumbasistemas/corubafood/security/PasswordHash.java](#) (1038B)
- [src/main/java/com/corumbasistemas/corubafood/test/TesteMesaProntaCompleto.java](#) (17862B)
- [src/main/java/com/corumbasistemas/corubafood/util/JPAUtil.java](#) (1497B)
- [src/main/java/com/corumbasistemas/corubafood/util/SeedData.java](#) (3152B)
- [src/main/java/com/corumbasistemas/corubafood/util/SeedMassivo.java](#) (11150B)
- [src/main/resources/META-INF/persistence.xml](#) (2014B)
- [src/main/resources/logback.xml](#) (1003B)
- [src/main/resources/view/ConfigIntegracaoView.fxml](#) (10388B)
- [src/main/resources/view/DeliveryView.fxml](#) (7262B)
- [src/main/resources/view/LoginView.fxml](#) (1897B)
- [src/main/resources/view/PrincipalView.fxml](#) (3688B)

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
3   <modelVersion>4.0.0</modelVersion>
4   <groupId>com.corumbasistemas</groupId>
5   <artifactId>coruba-food</artifactId>
6   <name>Corumbá Food - MesaPronta</name>
7   <version>2.0.0</version>
8   <build>
9     <plugins>
10       <plugin>
11         <artifactId>maven-compiler-plugin</artifactId>
12         <version>3.12.1</version>
13         <configuration>
14           <source>17</source>
15           <target>17</target>
16         </configuration>
17       </plugin>
18       <plugin>
19         <artifactId>maven-shade-plugin</artifactId>
20         <version>3.5.2</version>
21         <executions>
22           <execution>
23             <phase>package</phase>
24             <goals>
25               <goal>shade</goal>
26             </goals>
27             <configuration>
28               <transformers>
29                 <transformer>
30                   <mainClass>com.corumbasistemas.corubafood.CorubaFoodApp</mainClass>
31                 </transformer>
32               </transformers>
33             </configuration>
34           </execution>
35         </executions>
36       </plugin>
37       <plugin>
38         <groupId>org.openjfx</groupId>
39         <artifactId>javafx-maven-plugin</artifactId>
40         <version>0.0.8</version>
41         <configuration>
42           <mainClass>com.corumbasistemas.corubafood.CorubaFoodApp</mainClass>
43         </configuration>
44       </plugin>
45       <plugin>
46         <artifactId>maven-surefire-plugin</artifactId>
47         <version>3.2.5</version>
48       </plugin>
49     </plugins>
50   </build>
51   <dependencies>
52     <dependency>
53       <groupId>org.junit.jupiter</groupId>
54       <artifactId>junit-jupiter</artifactId>
55       <version>5.10.2</version>
56       <scope>test</scope>
57       <exclusions>
58         <exclusion>
59           <artifactId>junit-jupiter-api</artifactId>
60           <groupId>org.junit.jupiter</groupId>
61         </exclusion>
62         <exclusion>
63           <artifactId>junit-jupiter-params</artifactId>
64           <groupId>org.junit.jupiter</groupId>
65         </exclusion>
66         <exclusion>
67           <artifactId>junit-jupiter-engine</artifactId>
68           <groupId>org.junit.jupiter</groupId>
69         </exclusion>
70       </exclusions>
71     </dependency>
72   </dependencies>
73   <properties>
74     <hibernate.version>6.4.4.Final</hibernate.version>
75     <maven.compiler.source>17</maven.compiler.source>
76     <jbcrypt.version>0.4</jbcrypt.version>
77     <maven.compiler.target>17</maven.compiler.target>
78     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
79     <javafx.version>21</javafx.version>
80     <logback.version>1.5.3</logback.version>
81     <sqlite.version>3.45.1.0</sqlite.version>
82   </properties>
83 </project>
84

```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6   <groupId>com.corumbasistemas</groupId>
7   <artifactId>coruba-food-jdk21</artifactId>
8   <version>2.1.0</version>
9   <packaging>jar</packaging>
10  <name>Corumbá Food - MesaPronta (JDK 21)</name>
11
12  <properties>
13    <maven.compiler.source>21</maven.compiler.source>
14    <maven.compiler.target>21</maven.compiler.target>
15    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
16    <javafx.version>21.0.8</javafx.version>
17    <hibernate.version>6.6.4.Final</hibernate.version>
18    <sqlite.version>3.49.1.0</sqlite.version>
19    <jbcrypt.version>0.4</jbcrypt.version>
20    <logback.version>1.5.18</logback.version>
21    <slf4j.version>2.0.17</slf4j.version>
22  </properties>
23
24  <dependencies>
25    <!-- JavaFX 21 -->
26    <dependency>
27      <groupId>org.openjfx</groupId>
28      <artifactId>javafx-controls</artifactId>
29      <version>${javafx.version}</version>
30    </dependency>
31    <dependency>
32      <groupId>org.openjfx</groupId>
33      <artifactId>javafx-fxml</artifactId>
34      <version>${javafx.version}</version>
35    </dependency>
36    <dependency>
37      <groupId>org.openjfx</groupId>
38      <artifactId>javafx-graphics</artifactId>
39      <version>${javafx.version}</version>
40    </dependency>
41    <!-- Hibernate 6.6 (compatível com JDK 21) -->
42    <dependency>
43      <groupId>org.hibernate.orm</groupId>
44      <artifactId>hibernate-core</artifactId>
45      <version>${hibernate.version}</version>
46    </dependency>
47    <dependency>
48      <groupId>org.hibernate.orm</groupId>
49      <artifactId>hibernate-c3p0</artifactId>
50      <version>${hibernate.version}</version>
51    </dependency>
52    <dependency>
53      <groupId>org.xerial</groupId>
54      <artifactId>sqlite-jdbc</artifactId>
55      <version>${sqlite.version}</version>
56    </dependency>
57    <dependency>
58      <groupId>org.hibernate.orm</groupId>
59      <artifactId>hibernate-community-dialects</artifactId>
60      <version>${hibernate.version}</version>
61    </dependency>
62    <dependency>
63      <groupId>org.mindrot</groupId>
64      <artifactId>jbcrypt</artifactId>
65      <version>${jbcrypt.version}</version>
66    </dependency>
67    <dependency>
68      <groupId>ch.qos.logback</groupId>
69      <artifactId>logback-classic</artifactId>
70      <version>${logback.version}</version>
71    </dependency>
72    <dependency>
73      <groupId>org.slf4j</groupId>
74      <artifactId>slf4j-api</artifactId>
75      <version>${slf4j.version}</version>
76    </dependency>
77    <dependency>
78      <groupId>org.junit.jupiter</groupId>
79      <artifactId>junit-jupiter</artifactId>
80      <version>5.12.0</version>
81      <scope>test</scope>
82    </dependency>
83  </dependencies>
84
85  <build>
86    <plugins>
87      <plugin>
88        <groupId>org.apache.maven.plugins</groupId>
89        <artifactId>maven-compiler-plugin</artifactId>
90        <version>3.13.0</version>
91        <configuration>
92          <source>21</source>
93          <target>21</target>
94          <release>21</release>
95        </configuration>
96      </plugin>
97      <plugin>
98        <groupId>org.openjfx</groupId>
99        <artifactId>javafx-maven-plugin</artifactId>
100       <version>0.0.8</version>
101       <configuration>
102         <mainClass>com.corumbasistemas.corubafood.CorubaFoodApp</mainClass>
103         <options>
104           <option>-add-opens</option>
105           <option>javafx.graphics/com.sun.javafx.application=ALL-UNNAMED</option>
106         </options>
107       </configuration>
108     </plugin>
109   </plugins>
110 </build>
111 </project>
112
```

```
1
2  TESTE COMPLETO MESAPRONTA v2.0
3  Corumbá Sistemas
4
5
6  ✓ PASSOU [1062ms] Seed Demo
7  ✓ PASSOU [334798ms] Seed Massivo (100 empresas)
8  ✓ PASSOU [ 1ms] Contagem: 100+ empresas
9  ✓ PASSOU [ 1ms] Contagem: 1000+ usuários
10 ✓ PASSOU [ 1ms] Contagem: 10000+ produtos
11 ✓ PASSOU [ 1ms] Contagem: 1000+ mesas
12 ✓ PASSOU [ 1ms] Contagem: 500+ categorias
13 ✓ PASSOU [ 361ms] Login admin/demo
14 ✓ PASSOU [ 332ms] Login senha errada rejeita
15 ✓ PASSOU [ 331ms] Login garçom/demo
16 ✓ PASSOU [ 42ms] CRUD Produto - buscar todos
17 ✓ PASSOU [ 12ms] CRUD Produto - buscar por categoria
18 ✓ PASSOU [ 6ms] CRUD Usuario - buscar todos
19 ✓ PASSOU [ 53ms] Criar pedido completo
20 ✓ PASSOU [ 15ms] Criar pagamento
21 ✓ PASSOU [ 986ms] BCrypt - hash e verify
22 ✓ PASSOU [ 0ms] BCrypt - migração plaintext
23 ✓ PASSOU [ 2ms] Buscar empresa por CNPJ
24 ✓ PASSOU [ 1ms] Empresa inexistente retorna null
25 ✓ PASSOU [ 660ms] Verificar isAdmin
26 ✓ PASSOU [ 22ms] Performance: listar 100 empresas < 1s
27 ✓ PASSOU [ 1ms] Performance: contar 10000+ produtos < 1s
28 ✓ PASSOU [ 12ms] Integridade: empresa tem 10+ usuários
29 ✓ PASSOU [ 9ms] Integridade: empresa tem 5+ produtos
30 ✓ PASSOU [ 12ms] Integridade: empresa tem 10 mesas
31
32
33  RESULTADO FINAL
34
35  ✓ Passaram: 25
36  ✗ Falharam: 0
37  📊 Total: 25
38  ⌚ Tempo: 340684ms (340.7s)
39
40
41  🐞 TODOS OS TESTES PASSARAM! 🐞
42
```

```

1 package com.corumbasistemas.corubafood;
2
3 import com.corumbasistemas.corubafood.controller.AuthController;
4 import com.corumbasistemas.corubafood.model.Usuario;
5 import com.corumbasistemas.corubafood.util.*;
6 import javafx.application.Application;
7 import javafx.fxml.FXMLLoader;
8 import javafx.scene.*;
9 import javafx.stage.Stage;
10 import org.slf4j.*;
11
12 /**
13  * CorubaFoodApp - MesaPronta v2.1 (JDK 21)
14  *
15  * Novidades JDK 21:
16  * - var para inferência de tipo local
17  * - Text blocks para strings multilinha
18  * - Pattern matching em switch
19  * - Records para DTOs
20  * - Formatted strings
21  * - Sealed classes
22  */
23 public class CorubaFoodApp extends Application {
24     private static final Logger logger = LoggerFactory.getLogger(CorubaFoodApp.class);
25     private static Stage ps;
26     private static AuthController ac;
27     private static Usuario ul;
28
29     @Override
30     public void start(Stage s) throws Exception {
31         ps = s;
32         ac = new AuthController();
33         JPAUtil.getEMF();
34
35         // Text block (JDK 21)
36         logger.info("""
37
38             Coruba Food - MesaPronta v2.1 (JDK 21)
39             """);
40
41         // Tenta seed normal primeiro, se vazio faz massivo
42         try {
43             SeedData.popular();
44         } catch (Exception e) {
45             logger.warn("Seed normal: {}", e.getMessage());
46         }
47
48         try {
49             SeedMassivo.popular();
50         } catch (Exception e) {
51             logger.warn("Seed massivo: {}", e.getMessage());
52         }
53
54         mostrarLogin();
55     }
56
57     public static void mostrarLogin() throws Exception {
58         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/LoginView.fxml"));
59         Parent r = l.load();
60         ps.setTitle("Coruba Food - Login");
61         ps.setScene(new Scene(r));
62         ps.setResizable(false);
63         ps.show();
64     }
65
66     public static void mostrarPrincipal() throws Exception {
67         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/PrincipalView.fxml"));
68         Parent r = l.load();
69
70         // Formatted string (JDK 21)
71         var titulo = "Coruba Food - MesaPronta v2.1 [%s]".formatted(ul.getNome());
72         ps.setTitle(titulo);
73         ps.setScene(new Scene(r, 1200, 800));
74         ps.setResizable(true);
75         ps.setMaximized(true);
76         ps.show();
77     }
78
79     public static void mostrarDelivery() throws Exception {
80         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/DeliveryView.fxml"));
81         Parent r = l.load();
82
83         var titulo = "Coruba Food - ConnectFood (iFood/99Food) [%s]".formatted(ul.getNome());
84         ps.setTitle(titulo);
85         ps.setScene(new Scene(r, 1200, 800));
86         ps.setResizable(true);
87         ps.setMaximized(true);
88         ps.show();
89     }
90
91     public static void mostrarConfigIntegracao() throws Exception {
92         var l = new FXMLLoader(CorubaFoodApp.class.getResource("/view/ConfigIntegracaoView.fxml"));
93         Parent r = l.load();
94
95         var titulo = "Coruba Food - Configuração APIs [%s]".formatted(ul.getNome());
96         ps.setTitle(titulo);
97         ps.setScene(new Scene(r, 700, 700));
98         ps.setResizable(true);
99         ps.show();
100     }
101
102     public static Stage getPS() { return ps; }
103     public static AuthController getAuthController() { return ac; }
104     public static Usuario getUsuarioLogado() { return ul; }
105     public static void setUsuarioLogado(Usuario u) { ul = u; }
106
107     @Override
108     public void stop() {
109         JPAUtil.shutdown();
110         logger.info("Coruba Food parado");
111     }
112
113     public static void main(String... args) {
114         try {
115             launch(args);
116         } catch (IllegalStateException e) {
117             System.err.println("JavaFX Runtime already started: " + e.getMessage());
118         } catch (Exception e) {
119             System.err.println("Erro ao iniciar aplicacao: " + e.getMessage());
120             e.printStackTrace();
121             System.exit(1);
122         }
123     }
124 }
125
126

```

```
1 package com.corumbasistemas.corubafood.controller;
2 import com.corumbasistemas.corubafood.model.*;
3 import com.corumbasistemas.corubafood.repository.UsuarioRepository;
4 import com.corumbasistemas.corubafood.security.PasswordHash;
5 import com.corumbasistemas.corubafood.util.JPAUtil;
6 import jakarta.persistence.*;
7 import org.slf4j.*;
8 public class AuthController {
9     private static final Logger logger = LoggerFactory.getLogger(AuthController.class);
10    private final UsuarioRepository repo = new UsuarioRepository();
11    public Usuario autenticar(Long eid, String u, String s) {
12        try {
13            Usuario usr = repo.buscarPorUsername(eid, u);
14            if (usr == null) { PasswordHash.hash("dummy"); logger.warn("Login invalid: {}", u); return null; }
15            if (!PasswordHash.verify(s, usr.getSenha())) { logger.warn("Wrong password: {}", u); return null; }
16            if (PasswordHash.needsRehash(usr.getSenha())) { logger.info("Migrating password for {}", u); usr.setSenha(PasswordHash.hash(s)); repo.salvar(usr); }
17            logger.info("Login OK: {}", u); return usr;
18        } catch (Exception e) { logger.error("Auth error: {}", e.getMessage(), e); return null; }
19    }
20    public Empresa buscarEmpresaPorDocumento(String doc) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT e FROM Empresa e WHERE e.documento=:d AND e.ativo=true", Empresa.class).setParameter("d", doc).getSingleResult(); } catch (NoResultException e) { return null; } finally { em.close(); } }
21    public boolean isAdmin(Usuario u) { return u != null && u.getPapel() == Usuario.Papel.ADMIN; }
22 }
```

```

1 package com.corumbasistemas.corubafood.controller;
2
3 import javafx.fxml.FXML;
4 import javafx.fxml.Initializable;
5 import javafx.scene.control.*;
6 import javafx.scene.layout.VBox;
7 import javafx.scene.paint.Color;
8 import javafx.scene.text.Font;
9 import javafx.scene.text.FontWeight;
10 import org.slf4j.*;
11
12 import java.io.*;
13 import java.net.URL;
14 import java.util.Properties;
15 import java.util.ResourceBundle;
16
17 /**
18  * ConfigIntegracaoController - Tela de configuração das APIs.
19  *
20  * 0 cliente cola as credenciais da iFood e 99Food aqui.
21  * As credenciais são salvas em um arquivo .properties local.
22  *
23  * Como obter as credenciais:
24  *
25  * iFood:
26  * 1. Acesse developer.ifood.com.br
27  * 2. Registre-se como parceiro merchant
28  * 3. Crie um app para obter client_id e client_secret
29  * 4. Copie o merchant_id do seu estabelecimento
30  *
31  * 99Food:
32  * 1. Acesse o painel do parceiro 99Food
33  * 2. Solicite acesso à API pelo suporte
34  * 3. Receba client_id, client_secret e merchant_id
35  */
36 public class ConfigIntegracaoController implements Initializable {
37     private static final Logger logger = LoggerFactory.getLogger(ConfigIntegracaoController.class);
38     private static final String CONFIG_FILE = "integrations.properties";
39
40     // ===== iFood =====
41     @FXML private TextField txtIfoodClientId;
42     @FXML private PasswordField txtIfoodClientSecret;
43     @FXML private TextField txtIfoodMerchantId;
44     @FXML private Label lblIfoodStatus;
45     @FXML private Button btnIfoodTestar;
46     @FXML private Button btnIfoodSalvar;
47
48     // ===== 99Food =====
49     @FXML private TextField txt99ClientId;
50     @FXML private PasswordField txt99ClientSecret;
51     @FXML private TextField txt99MerchantId;
52     @FXML private Label lbl99Status;
53     @FXML private Button btn99Testar;
54     @FXML private Button btn99Salvar;
55
56     // ===== Geral =====
57     @FXML private Label lblStatusBar;
58     @FXML private Spinner<Integer> spnPollInterval;
59     @FXML private CheckBox chkAutoStart;
60     @FXML private CheckBox chkSoundAlert;
61
62     private Properties config;
63
64     @Override
65     public void initialize(URL url, ResourceBundle rb) {
66         config = new Properties();
67         loadConfig();
68
69         // Configurar spinner de intervalo (10-300 segundos)
70         SpinnerValueFactory<Integer> factory = new SpinnerValueFactory.IntegerSpinnerValueFactory(
71             10, 300, Integer.parseInt(config.getProperty("sync.poll_interval", "30"))
72         );
73         spnPollInterval.setValueFactory(factory);
74
75         // Preencher campos iFood
76         txtIfoodClientId.setText(config.getProperty("ifood.client_id", ""));
77         txtIfoodClientSecret.setText(config.getProperty("ifood.client_secret", ""));
78         txtIfoodMerchantId.setText(config.getProperty("ifood.merchant_id", ""));
79
80         // Preencher campos 99Food
81         txt99ClientId.setText(config.getProperty("food99.client_id", ""));
82         txt99ClientSecret.setText(config.getProperty("food99.client_secret", ""));
83         txt99MerchantId.setText(config.getProperty("food99.merchant_id", ""));
84
85         // Checkboxes
86         chkAutoStart.setSelected("true".equals(config.getProperty("sync.auto_start", "true")));
87         chkSoundAlert.setSelected("true".equals(config.getProperty("sync.sound_alert", "true")));
88
89         updateStatusLabel();
90         lblStatusBar.setText("Configuração carregada. Preencha as credenciais e clique em Testar.");
91     }
92
93     /**
94      * Salva configurações do iFood
95      */
96     @FXML
97     private void handleIfoodSalvar() {
98         config.setProperty("ifood.client_id", txtIfoodClientId.getText().trim());
99         config.setProperty("ifood.client_secret", txtIfoodClientSecret.getText().trim());
100        config.setProperty("ifood.merchant_id", txtIfoodMerchantId.getText().trim());
101        saveConfig();
102        lblStatusBar.setText("✓ Credenciais iFood salvas!");
103        logger.info("Config iFood salva (merchant: {})", txtIfoodMerchantId.getText().trim());
104    }
105
106    /**
107     * Configurações do 99Food
108     */
109    @FXML
110    private void handle99Salvar() {
111        config.setProperty("food99.client_id", txt99ClientId.getText().trim());
112        config.setProperty("food99.client_secret", txt99ClientSecret.getText().trim());
113        config.setProperty("food99.merchant_id", txt99MerchantId.getText().trim());
114        saveConfig();
115        lblStatusBar.setText("✓ Credenciais 99Food salvas!");
116        logger.info("Config 99Food salva (merchant: {})", txt99MerchantId.getText().trim());
117    }
118
119    /**
120     * Testa conexão com iFood
121     * (Em produção, faz uma chamada real de API)
122     */
123    @FXML
124    private void handleIfoodTestar() {
125        String clientId = txtIfoodClientId.getText().trim();
126        String clientSecret = txtIfoodClientSecret.getText().trim();
127        String merchantId = txtIfoodMerchantId.getText().trim();
128
129        if (clientId.isEmpty() || clientSecret.isEmpty() || merchantId.isEmpty()) {
130            lblIfoodStatus.setTextFill(Color.ORANGE);
131            lblIfoodStatus.setText("⚠ Preencha todos os campos");
132            return;
133        }
134    }

```



```

135     lblIfoodStatus.setText("❌ Testando conexão...");
136     lblIfoodStatus.setTextFill(Color.BLUE);
137
138     // Simula teste de conexão
139     new Thread(() -> {
140         try {
141             Thread.sleep(1500); // Simula latência
142
143             // Validação básica de formato
144             boolean valid = clientId.length() >= 10 && clientSecret.length() >= 10;
145
146             javafx.application.Platform.runLater(() -> {
147                 if (valid) {
148                     lblIfoodStatus.setTextFill(Color.GREEN);
149                     lblIfoodStatus.setText("✓ Credenciais válidas! Conexão OK.");
150                     lblStatusBar.setText("iFood: conexão testada com sucesso. Merchant: " + merchantId);
151                 } else {
152                     lblIfoodStatus.setTextFill(Color.RED);
153                     lblIfoodStatus.setText("✗ Credenciais inválidas. Verifique e tente novamente.");
154                 }
155             });
156         } catch (InterruptedException e) {
157             javafx.application.Platform.runLater(() -> {
158                 lblIfoodStatus.setTextFill(Color.RED);
159                 lblIfoodStatus.setText("✗ Erro no teste");
160             });
161         }
162     }).start();
163 }
164
165 /**
166  * Testa conexão com 99Food
167  */
168 @FXML
169 private void handle99Testar() {
170     String clientId = txt99ClientId.getText().trim();
171     String clientSecret = txt99ClientSecret.getText().trim();
172     String merchantId = txt99MerchantId.getText().trim();
173
174     if (clientId.isEmpty() || clientSecret.isEmpty() || merchantId.isEmpty()) {
175         lbl99Status.setTextFill(Color.ORANGE);
176         lbl99Status.setText("⚠ Preencha todos os campos");
177         return;
178     }
179
180     lbl99Status.setText("❌ Testando conexão...");
181     lbl99Status.setTextFill(Color.BLUE);
182
183     new Thread(() -> {
184         try {
185             Thread.sleep(1500);
186             boolean valid = clientId.length() >= 10 && clientSecret.length() >= 10;
187
188             javafx.application.Platform.runLater(() -> {
189                 if (valid) {
190                     lbl99Status.setTextFill(Color.GREEN);
191                     lbl99Status.setText("✓ Credenciais válidas! Conexão OK.");
192                     lblStatusBar.setText("99Food: conexão testada com sucesso. Merchant: " + merchantId);
193                 } else {
194                     lbl99Status.setTextFill(Color.RED);
195                     lbl99Status.setText("✗ Credenciais inválidas. Verifique e tente novamente.");
196                 }
197             });
198         } catch (InterruptedException e) {
199             javafx.application.Platform.runLater(() -> {
200                 lbl99Status.setTextFill(Color.RED);
201                 lbl99Status.setText("✗ Erro no teste");
202             });
203         }
204     }).start();
205 }
206
207 /**
208  * Salva configurações gerais de sincronização
209  */
210 @FXML
211 private void handleSalvarGeral() {
212     config.setProperty("sync.poll_interval", spnPollInterval.getValue().toString());
213     config.setProperty("sync.auto_start", chkAutoStart.isSelected() ? "true" : "false");
214     config.setProperty("sync.sound_alert", chkSoundAlert.isSelected() ? "true" : "false");
215     saveConfig();
216     lblStatusBar.setText("✓ Configurações gerais salvas! Intervalo: " + spnPollInterval.getValue() + "s");
217 }
218
219 /**
220  * Volta para o delivery
221  */
222 @FXML
223 private void handleVoltar() {
224     try {
225         com.corumbasistemas.corubafood.CorubaFoodApp.mostrarDelivery();
226     } catch (Exception e) {
227         logger.error("Erro ao voltar: {}", e.getMessage());
228     }
229 }
230
231 // ===== Persistência =====
232
233 private void loadConfig() {
234     File f = new File(CONFIG_FILE);
235     if (f.exists()) {
236         try (FileInputStream fis = new FileInputStream(f)) {
237             config.load(fis);
238             logger.info("Config carregada de {}", CONFIG_FILE);
239         } catch (IOException e) {
240             logger.error("Erro ao carregar config: {}", e.getMessage());
241         }
242     }
243 }
244
245 private void saveConfig() {
246     try (FileOutputStream fos = new FileOutputStream(CONFIG_FILE)) {
247         config.store(fos, "ConnectFood - Configuração das APIs\nGerado em: " + new java.util.Date());
248         logger.info("Config salva em {}", CONFIG_FILE);
249     } catch (IOException e) {
250         logger.error("Erro ao salvar config: {}", e.getMessage());
251         lblStatusBar.setText("✗ Erro ao salvar: " + e.getMessage());
252     }
253 }
254
255 private void updateStatusLabels() {
256     boolean ifoodOk = !txtIfoodClientId.getText().trim().isEmpty()
257         && !txtIfoodClientSecret.getText().trim().isEmpty();
258     lblIfoodStatus.setText(ifoodOk ? "🟢 Configurado" : "🔴 Não configurado");
259     lblIfoodStatus.setTextFill(ifoodOk ? Color.GRAY : Color.GRAY);
260
261     boolean f99Ok = !txt99ClientId.getText().trim().isEmpty()
262         && !txt99ClientSecret.getText().trim().isEmpty();
263     lbl99Status.setText(f99Ok ? "🟢 Configurado" : "🔴 Não configurado");
264     lbl99Status.setTextFill(f99Ok ? Color.GRAY : Color.GRAY);
265 }
266 }
267

```

```

1 package com.corumbasistemas.corubafood.controller;
2
3 import com.corumbasistemas.corubafood.CorubaFoodApp;
4 import com.corumbasistemas.corubafood.model.*;
5 import com.corumbasistemas.corubafood.util.JPAUtil;
6 import javafx.collections.FXCollections;
7 import javafx.collections.ObservableList;
8 import javafx.fxml.FXML;
9 import javafx.fxml.Initializable;
10 import javafx.geometry.Insets;
11 import javafx.geometry.Pos;
12 import javafx.scene.control.*;
13 import javafx.scene.layout.*;
14 import javafx.scene.paint.Color;
15 import javafx.scene.text.Font;
16 import javafx.scene.text.FontWeight;
17 import org.slf4j.*;
18
19 import java.math.BigDecimal;
20 import java.net.URL;
21 import java.time.format.DateTimeFormatter;
22 import java.util.ResourceBundle;
23
24 /**
25  * DeliveryController - Gerencia pedidos de delivery (iFood e 99Food).
26  *
27  * Funcionalidades:
28  * - Listar pedidos recebidos de cada plataforma
29  * - Confirmar, preparar, despachar e cancelar pedidos
30  * - Visualizar detalhes do pedido (cliente, itens, valores)
31  * - Dashboard com contadores por status
32  */
33 public class DeliveryController implements Initializable {
34     private static final Logger logger = LoggerFactory.getLogger(DeliveryController.class);
35     private static final DateTimeFormatter FMT = DateTimeFormatter.ofPattern("HH:mm:ss");
36
37     // --- Componentes da Tela ---
38     @FXML private Label lblUsuario;
39     @FXML private Label lblStatusBar;
40     @FXML private Label lblTotalIfood;
41     @FXML private Label lblTotal99food;
42     @FXML private Label lblNovos;
43     @FXML private Label lblPreparando;
44     @FXML private Label lblProntos;
45     @FXML private ListView<String> lvPedidos;
46     @FXML private VBox vbDetalhes;
47     @FXML private Label lblDetalhesTitulo;
48     @FXML private TextArea txtDetalhesConteudo;
49     @FXML private ToggleButton btnAbaIfood;
50     @FXML private ToggleButton btnAba99food;
51     @FXML private ToggleButton btnAbaTodos;
52
53     private PedidoDelivery pedidoSelecioneado;
54     private PedidoDelivery.Plataforma plataformaFiltro = null; // null = todos
55     private final ObservableList<String> pedidosLista = FXCollections.observableArrayList();
56
57     @Override
58     public void initialize(URL url, ResourceBundle rb) {
59         Usuario u = CorubaFoodApp.getUsuarioLogado();
60         if (u != null) {
61             lblUsuario.setText(u.getNome() + " (" + u.getPapel() + ")");
62         }
63
64         lvPedidos.setItems(pedidosLista);
65         lvPedidos.getSelectionModel().selectedIndexProperty().addListener(
66             (obs, old, newIdx) -> mostrarDetalhes(newIdx.intValue())
67         );
68
69         // Configurar abas de filtro
70         configurarAbas();
71
72         // Carregar pedidos
73         carregarPedidos();
74         atualizarContadores();
75
76         lblStatusBar.setText("Delivery carregado - " + pedidosLista.size() + " pedidos");
77     }
78
79     private void configurarAbas() {
80         ToggleGroup grupo = new ToggleGroup();
81         btnAbaIfood.setToggleGroup(grupo);
82         btnAba99food.setToggleGroup(grupo);
83         btnAbaTodos.setToggleGroup(grupo);
84         btnAbaTodos.setSelected(true);
85
86         btnAbaIfood.setOnAction(e -> {
87             plataformaFiltro = PedidoDelivery.Plataforma.IF00D;
88             carregarPedidos();
89         });
90         btnAba99food.setOnAction(e -> {
91             plataformaFiltro = PedidoDelivery.Plataforma.F00D99;
92             carregarPedidos();
93         });
94         btnAbaTodos.setOnAction(e -> {
95             plataformaFiltro = null;
96             carregarPedidos();
97         });
98     }
99
100     /**
101      * Carrega pedidos do banco de dados, filtrando por plataforma se necessário
102      */
103     private void carregarPedidos() {
104         pedidosLista.clear();
105         Usuario u = CorubaFoodApp.getUsuarioLogado();
106         if (u == null) return;
107
108         try {
109             var em = JPAUtil.getEM();
110             String jpql = "SELECT p FROM PedidoDelivery p WHERE p.empresa.id = :eid";
111             if (plataformaFiltro != null) {
112                 jpql += " AND p.plataforma = :plat";
113             }
114             jpql += " ORDER BY p.dataCriacao DESC";
115
116             var query = em.createQuery(jpql, PedidoDelivery.class)
117                 .setParameter("eid", u.getEmpresa().getId());
118             if (plataformaFiltro != null) {
119                 query.setParameter("plat", plataformaFiltro);
120             }
121
122             var pedidos = query.getResultList();
123             em.close();
124
125             for (PedidoDelivery p : pedidos) {
126                 String iconePlataforma = p.getPlataforma() == PedidoDelivery.Plataforma.IF00D ? "🍷" : "🍷";
127                 String iconeStatus = iconeStatus(p.getStatus());
128                 String linha = String.format("%s %s | %s | %s | R$ %.2f",
129                     iconePlataforma, p.getCodigoPlataforma(),
130                     p.getNomeCliente(), iconeStatus, p.getValorTotal());
131                 pedidosLista.add(linha);
132             }
133
134             lblStatusBar.setText(pedidosLista.size() + " pedidos carregados");

```

```

135     } catch (Exception e) {
136         logger.error("Erro ao carregar pedidos delivery: {}", e.getMessage());
137         lblStatusBar.setText("Erro ao carregar pedidos");
138     }
139 }
140
141 /**
142  * Atualiza os contadores do dashboard
143  */
144 private void atualizarContadores() {
145     Usuario u = CorubaFoodApp.getUsuarioLogado();
146     if (u == null) return;
147
148     try {
149         var em = JPAUtil.getEM();
150
151         // Total iFood
152         Long totalIFood = em.createQuery(
153             "SELECT COUNT(p) FROM PedidoDelivery p WHERE p.empresa.id = :eid AND p.plataforma = :plat", Long.class)
154             .setParameter("eid", u.getEmpresa().getId())
155             .setParameter("plat", PedidoDelivery.Plataforma.IFOOD)
156             .getSingleResult();
157         lblTotalIFood.setText(totalIFood.toString());
158
159         // Total 99Food
160         Long total99 = em.createQuery(
161             "SELECT COUNT(p) FROM PedidoDelivery p WHERE p.empresa.id = :eid AND p.plataforma = :plat", Long.class)
162             .setParameter("eid", u.getEmpresa().getId())
163             .setParameter("plat", PedidoDelivery.Plataforma.FOOD99)
164             .getSingleResult();
165         lblTotal99Food.setText(total99.toString());
166
167         // Novos
168         Long novos = em.createQuery(
169             "SELECT COUNT(p) FROM PedidoDelivery p WHERE p.empresa.id = :eid AND p.status = :status", Long.class)
170             .setParameter("eid", u.getEmpresa().getId())
171             .setParameter("status", PedidoDelivery.StatusPedido.NOVO)
172             .getSingleResult();
173         lblNovos.setText(novos.toString());
174
175         // Preparando
176         Long prep = em.createQuery(
177             "SELECT COUNT(p) FROM PedidoDelivery p WHERE p.empresa.id = :eid AND p.status = :status", Long.class)
178             .setParameter("eid", u.getEmpresa().getId())
179             .setParameter("status", PedidoDelivery.StatusPedido.PREPARANDO)
180             .getSingleResult();
181         lblPreparando.setText(prepare.toString());
182
183         // Prontos
184         Long prontos = em.createQuery(
185             "SELECT COUNT(p) FROM PedidoDelivery p WHERE p.empresa.id = :eid AND p.status = :status", Long.class)
186             .setParameter("eid", u.getEmpresa().getId())
187             .setParameter("status", PedidoDelivery.StatusPedido.PRONTO)
188             .getSingleResult();
189         lblProntos.setText(prontos.toString());
190
191         em.close();
192     } catch (Exception e) {
193         logger.error("Erro ao atualizar contadores: {}", e.getMessage());
194     }
195 }
196
197 /**
198  * Mostra detalhes do pedido selecionado
199  */
200 private void mostrarDetalhes(int index) {
201     if (index < 0 || index >= pedidosLista.size()) return;
202
203     Usuario u = CorubaFoodApp.getUsuarioLogado();
204     if (u == null) return;
205
206     try {
207         var em = JPAUtil.getEM();
208         String jpql = "SELECT p FROM PedidoDelivery p WHERE p.empresa.id = :eid ORDER BY p.dataCriacao DESC";
209         var pedidos = em.createQuery(jpql, PedidoDelivery.class)
210             .setParameter("eid", u.getEmpresa().getId())
211             .getResultList();
212         em.close();
213
214         if (index < pedidos.size()) {
215             pedidoSelecionado = pedidos.get(index);
216             StringBuilder sb = new StringBuilder();
217             sb.append("ð PEDIDO: ").append(pedidoSelecionado.getCodigoPlataforma()).append("\n");
218             sb.append("-----\n");
219             sb.append("ð Plataforma: ").append(pedidoSelecionado.getPlataforma()).append("\n");
220             sb.append("ð Cliente: ").append(pedidoSelecionado.getNomeCliente()).append("\n");
221             sb.append("ð Telefone: ").append(nvl(pedidoSelecionado.getTelefoneCliente()).append("\n");
222             sb.append("ð Endereço: ").append(nvl(pedidoSelecionado.getEnderecoEntrega()).append("\n");
223             sb.append("ð Pagamento: ").append(nvl(pedidoSelecionado.getFormaPagamento()).append("\n");
224             sb.append("ð Recebido: ").append(pedidoSelecionado.getDataCriacao().format(FMT)).append("\n");
225             sb.append("ð Status: ").append(pedidoSelecionado.getStatus()).append("\n");
226             sb.append("ð ITENS:\n");
227             sb.append("-----\n");
228             if (pedidoSelecionado.getItems() != null) {
229                 for (var item : pedidoSelecionado.getItems()) {
230                     sb.append(String.format(" %dx %s\n", item.getQuantidade(), item.getNomeProduto()));
231                     sb.append(String.format("      R$ %.2f → R$ %.2f\n",
232                         item.getPrecoUnitario(), item.getSubtotal()));
233                     if (item.getObservacoes() != null && !item.getObservacoes().isEmpty()) {
234                         sb.append("      ").append(item.getObservacoes()).append("\n");
235                     }
236                 }
237             }
238             sb.append("-----\n");
239             sb.append(String.format("Subtotal: R$ %.2f\n", pedidoSelecionado.getValorItens()));
240             sb.append(String.format("Tx. Entrega: R$ %.2f\n",
241                 pedidoSelecionado.getTaxaEntrega() != null ? pedidoSelecionado.getTaxaEntrega() : BigDecimal.ZERO));
242             sb.append(String.format("TOTAL: R$ %.2f\n", pedidoSelecionado.getValorTotal()));
243             if (pedidoSelecionado.getObservacoes() != null && !pedidoSelecionado.getObservacoes().isEmpty()) {
244                 sb.append("\nð Observações: ").append(pedidoSelecionado.getObservacoes()).append("\n");
245             }
246
247             txtDetalhesConteudo.setText(sb.toString());
248             lblDetalhesTitulo.setText("Detalhes - " + pedidoSelecionado.getCodigoPlataforma());
249         }
250     } catch (Exception e) {
251         logger.error("Erro ao mostrar detalhes: {}", e.getMessage());
252     }
253 }
254
255 /**
256  * Avança o status do pedido selecionado
257  */
258 @FXML
259 private void handleAvancarStatus() {
260     if (pedidoSelecionado == null) {
261         lblStatusBar.setText("ð Selecione um pedido primeiro!");
262         return;
263     }
264
265     try {
266         var em = JPAUtil.getEM();
267         em.getTransaction().begin();
268         var pedido = em.find(PedidoDelivery.class, pedidoSelecionado.getId());
269         boolean avancou = pedido.avancarStatus();
270         em.merge(pedido);
271         em.getTransaction().commit();
272         em.close();

```

```

273         if (avancou) {
274             lblStatusBar.setText("⚡ Pedido " + pedido.getCodigoPlataforma() +
275                 " → " + pedido.getStatus());
276             carregarPedidos();
277             atualizarContadores();
278         } else {
279             lblStatusBar.setText("⚠ Pedido já está no status final");
280         }
281     } catch (Exception e) {
282         logger.error("Erro ao avançar status: {}", e.getMessage());
283         lblStatusBar.setText("✖ Erro ao avançar status");
284     }
285 }
286
287 /**
288  * Cancela o pedido selecionado
289  */
290 @FXML
291 private void handleCancelarPedido() {
292     if (pedidoSelecioneado == null) {
293         lblStatusBar.setText("⚠ Selecione um pedido primeiro!");
294         return;
295     }
296
297     TextInputDialog dialog = new TextInputDialog();
298     dialog.setTitle("Cancelar Pedido");
299     dialog.setHeaderText("Cancelar pedido " + pedidoSelecioneado.getCodigoPlataforma());
300     dialog.setContentText("Motivo do cancelamento:");
301
302     var result = dialog.showAndWait();
303     if (result.isPresent() && !result.get().trim().isEmpty()) {
304         try {
305             var em = JPAUtil.getEM();
306             em.getTransaction().begin();
307             var pedido = em.find(PedidoDelivery.class, pedidoSelecioneado.getId());
308             pedido.cancelar(result.get().trim());
309             em.merge(pedido);
310             em.getTransaction().commit();
311             em.close();
312
313             lblStatusBar.setText("🗑 Pedido " + pedido.getCodigoPlataforma() + " cancelado");
314             carregarPedidos();
315             atualizarContadores();
316         } catch (Exception e) {
317             logger.error("Erro ao cancelar pedido: {}", e.getMessage());
318             lblStatusBar.setText("✖ Erro ao cancelar pedido");
319         }
320     }
321 }
322
323 /**
324  * Simula a chegada de um novo pedido (para demonstração)
325  */
326 @FXML
327 private void handleSimularPedido() {
328     Usuario u = CorubaFoodApp.getUsuarioLogado();
329     if (u == null) return;
330
331     try {
332         var em = JPAUtil.getEM();
333         em.getTransaction().begin();
334
335         // Alterna entre iFood e 99Food
336         PedidoDelivery.Plataforma plat = (Math.random() > 0.5)
337             ? PedidoDelivery.Plataforma.IF000
338             : PedidoDelivery.Plataforma.F00D99;
339
340         String[] clientes = {"Maria Silva", "João Santos", "Ana Oliveira", "Carlos Souza",
341             "Pedro Lima", "Juliana Costa", "Lucas Ferreira", "Beatriz Almeida"};
342         String[] produtos = {"X-Burger", "X-Tudo", "X-Salada", "X-Bacon",
343             "X-Frango", "X-Calabresa", "Misto Quente", "Cachorro Quente"};
344
345         String codigo = (plat == PedidoDelivery.Plataforma.IF000 ? "IF" : "99F") +
346             System.currentTimeMillis() % 100000;
347         String cliente = clientes[(int)(Math.random() * clientes.length)];
348
349         PedidoDelivery pedido = new PedidoDelivery(plat, codigo, cliente, u.getEmpresa());
350         pedido.setTelefoneCliente("(62) 99999-" + (int)(Math.random() * 9000 + 1000));
351         pedido.setEnderecoEntrega("Rua " + (int)(Math.random() * 100 + 1) + " nº " + (int)(Math.random() * 500 + 1));
352         pedido.setFormaPagamento(Math.random() > 0.5 ? "PIX" : "Cartão de Crédito");
353         pedido.setTaxaEntrega(BigDecimal.valueOf(Math.random() * 10 + 5).setScale(2, BigDecimal.ROUND_HALF_UP));
354
355         // Adiciona 1-3 itens
356         int numItens = (int)(Math.random() * 3) + 1;
357         BigDecimal totalItens = BigDecimal.ZERO;
358         for (int i = 0; i < numItens; i++) {
359             String prod = produtos[(int)(Math.random() * produtos.length)];
360             int qtd = (int)(Math.random() * 3) + 1;
361             BigDecimal preco = BigDecimal.valueOf(Math.random() * 20 + 10).setScale(2, BigDecimal.ROUND_HALF_UP);
362             ItemPedidoDelivery item = new ItemPedidoDelivery(prod, qtd, preco);
363             pedido.getItens().add(item);
364             totalItens = totalItens.add(item.getSubtotal());
365         }
366
367         pedido.setValorItens(totalItens);
368         pedido.calcularTotal();
369
370         em.persist(pedido);
371         em.getTransaction().commit();
372         em.close();
373
374         lblStatusBar.setText("📄 Novo pedido " + plat + ": " + codigo + " - " + cliente);
375         carregarPedidos();
376         atualizarContadores();
377     } catch (Exception e) {
378         logger.error("Erro ao simular pedido: {}", e.getMessage());
379         lblStatusBar.setText("✖ Erro ao simular pedido");
380     }
381 }
382
383 /**
384  * Abre tela de configuração das APIs
385  */
386 @FXML
387 private void handleConfigurar() {
388     try {
389         CorubaFoodApp.mostrarConfigIntegracao();
390     } catch (Exception e) {
391         logger.error("Erro ao abrir config: {}", e.getMessage());
392     }
393 }
394
395 /**
396  * Volta para a tela principal
397  */
398 @FXML
399 private void handleVoltar() {
400     try {
401         CorubaFoodApp.mostrarPrincipal();
402     } catch (Exception e) {
403         logger.error("Erro ao voltar: {}", e.getMessage());
404     }
405 }
406
407 // --- Utilitários ---
408
409 private String iconeStatus(PedidoDelivery.StatusPedido status) {

```

```
411         return switch (status) {
412             case NOVO -> "📄 Novo";
413             case CONFIRMADO -> "✅ Confirmado";
414             case PREPARANDO -> "🔪🔪 Preparando";
415             case PRONTO -> "🍗 Pronto";
416             case SAIU_ENTREGA -> "🚚 Entrega";
417             case ENTREGUE -> "✓ Entregue";
418             case CANCELADO -> "❌ Cancelado";
419         };
420     }
421
422     private String nvl(String s) {
423         return (s != null && !s.isEmpty()) ? s : "-";
424     }
425 }
426
```

```
1 package com.corumbasistemas.corubafood.controller;
2 import com.corumbasistemas.corubafood.CorubaFoodApp;
3 import com.corumbasistemas.corubafood.model.*;
4 import javafx.fxml.FXML;
5 import javafx.scene.control.*;
6 import javafx.scene.paint.Color;
7 import org.slf4j.*;
8 public class LoginController {
9     private static final Logger logger = LoggerFactory.getLogger(LoginController.class);
10    @FXML private TextField txtDocumento, txtUsername;
11    @FXML private PasswordField txtSenha;
12    @FXML private Label lblMensagem;
13    @FXML private Button btnEntrar;
14    @FXML private ProgressIndicator progress;
15    @FXML public void initialize() { lblMensagem.setText(""); progress.setVisible(false); }
16    @FXML private void handleEntrar() {
17        String doc = txtDocumento.getText().trim(), user = txtUsername.getText().trim(), senha = txtSenha.getText();
18        if (doc.isEmpty() || user.isEmpty() || senha.isEmpty()) { lblMensagem.setTextFill(Color.RED); lblMensagem.setText("Fill all fields!"); return; }
19        progress.setVisible(true); btnEntrar.setDisable(true);
20        Empresa e = CorubaFoodApp.getAuthController().buscarEmpresaPorDocumento(doc);
21        if (e == null) { lblMensagem.setTextFill(Color.RED); lblMensagem.setText("Company not found!"); progress.setVisible(false); btnEntrar.setDisable(false); return; }
22        Usuario u = CorubaFoodApp.getAuthController().autenticar(e.getId(), user, senha);
23        if (u == null) { lblMensagem.setTextFill(Color.RED); lblMensagem.setText("Invalid credentials!"); progress.setVisible(false); btnEntrar.setDisable(false); return; }
24        CorubaFoodApp.setUsuarioLogado(u); logger.info("Login: {}", user);
25        try { CorubaFoodApp.mostrarPrincipal(); } catch (Exception ex) { logger.error("Error: {}", ex.getMessage(), ex); }
26    }
27 }
```

```
1 package com.corumbasistemas.corubafood.controller;
2 import com.corumbasistemas.corubafood.model.*;
3 import com.corumbasistemas.corubafood.repository.PagamentoRepository;
4 import com.corumbasistemas.corubafood.security.PasswordHash;
5 import org.slf4j.*;
6 import java.math.BigDecimal;
7 public class PagamentoController {
8     private static final Logger logger = LoggerFactory.getLogger(PagamentoController.class);
9     private final PagamentoRepository repo = new PagamentoRepository();
10    private final AuthController auth;
11    public PagamentoController(AuthController auth) { this.auth = auth; }
12    public boolean aplicarDesconto(Long pid, Long eid, BigDecimal desc, String user, String senha) {
13        Usuario a = auth.autenticar(eid, user, senha);
14        if (a == null || !auth.isAdmin(a)) { logger.warn("Unauthorized discount: {}", user); return false; }
15        try { Pagamento p = repo.buscarPorPedido(eid, pid); if (p != null) { p.setDesconto(desc); p.setDescontoAutorizadoPor(PasswordHash.hash(user)); repo.salvar(p);
16            logger.info("Discount {} on order {} by {}", desc, pid, user); return true; } return false; }
17        catch (Exception e) { logger.error("Discount error: {}", e.getMessage(), e); return false; }
18    }
```

```

1 package com.corumbasistemas.corubafood.controller;
2
3 import com.corumbasistemas.corubafood.CorubaFoodApp;
4 import com.corumbasistemas.corubafood.model.*;
5 import com.corumbasistemas.corubafood.repository.ProdutoRepository;
6 import com.corumbasistemas.corubafood.repository.UsuarioRepository;
7 import com.corumbasistemas.corubafood.util.JPAUtil;
8 import javafx.collections.FXCollections;
9 import javafx.collections.ObservableList;
10 import javafx.fxml.FXML;
11 import javafx.fxml.Initializable;
12 import javafx.geometry.Insets;
13 import javafx.geometry.Pos;
14 import javafx.scene.control.*;
15 import javafx.scene.layout.*;
16 import javafx.scene.paint.Color;
17 import javafx.scene.text.Font;
18 import javafx.scene.text.FontWeight;
19 import org.slf4j.*;
20
21 import java.math.BigDecimal;
22 import java.net.URL;
23 import java.util.ResourceBundle;
24
25 /**
26  * PrincipalController - MesaPronta (JDK 21)
27  * Usa: var, pattern matching, switch expressions, text blocks, records
28  */
29 public class PrincipalController implements Initializable {
30     private static final Logger logger = LoggerFactory.getLogger(PrincipalController.class);
31
32     @FXML private Label lblUsuario;
33     @FXML private Label lblMesaTitulo;
34     @FXML private Label lblMesaStatus;
35     @FXML private Label lblTotal;
36     @FXML private Label lblStatusBar;
37     @FXML private ListView<String> lvItens;
38     @FXML private FlowPane fpMesas;
39
40     private final ProdutoRepository produtoRepo = new ProdutoRepository();
41     private Mesa mesaSelecionada;
42     private Pedido pedidoAtual;
43     private final ObservableList<String> itensLista = FXCollections.observableArrayList();
44
45     @Override
46     public void initialize(URL url, ResourceBundle rb) {
47         // var (JDK 21)
48         var u = CorubaFoodApp.getUsuarioLogado();
49         if (u != null) {
50             lblUsuario.setText("%s (%s)".formatted(u.getNome(), u.getPapel()));
51         }
52         lvItens.setItems(itensLista);
53         carregarMesas();
54         lblStatusBar.setText("Selecione uma mesa para começar");
55     }
56
57     private void carregarMesas() {
58         fpMesas.getChildren().clear();
59         var u = CorubaFoodApp.getUsuarioLogado();
60         if (u == null) return;
61
62         try {
63             var em = JPAUtil.getEM();
64             // Text block para query (JDK 21)
65             var query = """
66                 SELECT m FROM Mesa m
67                 WHERE m.empresa.id = :eid
68                 ORDER BY m.numero
69                 """;
70             var mesas = em.createQuery(query, Mesa.class)
71                 .setParameter("eid", u.getEmpresa().getId())
72                 .getResultList();
73             em.close();
74
75             for (var m : mesas) {
76                 var btn = new Button("Mesa %d\n%d lugares".formatted(m.getNumero(), m.getCapacidade()));
77                 btn.setPrefSize(120, 80);
78                 btn.setFont(Font.font(12));
79                 btn.setAlignment(Pos.CENTER);
80
81                 // Pattern matching com switch (JDK 21)
82                 switch (m.getStatus()) {
83                     case LIVRE -> {
84                         btn.setStyle("-fx-background-color: #4ec337; -fx-text-fill: #0f0f23; -fx-font-weight: bold;");
85                         btn.setOnAction(e -> selecionarMesa(m, btn));
86                     }
87                     case OCUPADA -> {
88                         btn.setStyle("-fx-background-color: #e9456b; -fx-text-fill: white; -fx-font-weight: bold;");
89                         btn.setOnAction(e -> selecionarMesa(m, btn));
90                     }
91                     case RESERVADA -> {
92                         btn.setStyle("-fx-background-color: #f77f00; -fx-text-fill: white; -fx-font-weight: bold;");
93                         btn.setOnAction(e -> selecionarMesa(m, btn));
94                     }
95                 }
96                 fpMesas.getChildren().add(btn);
97             }
98             lblStatusBar.setText("%d mesas carregadas".formatted(mesas.size()));
99         } catch (Exception e) {
100             logger.error("Erro ao carregar mesas: {}", e.getMessage());
101             lblStatusBar.setText("Erro ao carregar mesas");
102         }
103     }
104
105     private void selecionarMesa(Mesa mesa, Button btn) {
106         this.mesaSelecionada = mesa;
107         lblMesaTitulo.setText("Mesa %d (%d lugares)".formatted(mesa.getNumero(), mesa.getCapacidade()));
108         lblMesaStatus.setText(mesa.getStatus().name());
109
110         if (mesa.getStatus() == Mesa.Status.LIVRE) {
111             pedidoAtual = new Pedido();
112             pedidoAtual.setMesa(mesa);
113             pedidoAtual.setUsuario(CorubaFoodApp.getUsuarioLogado());
114             pedidoAtual.setEmpresa(mesa.getEmpresa());
115             pedidoAtual.setStatus(Pedido.Status.ABERTO);
116             itensLista.clear();
117             lblTotal.setText("R$ 0,00");
118             lblStatusBar.setText("Mesa %d - Novo pedido".formatted(mesa.getNumero()));
119         } else {
120             lblStatusBar.setText("Mesa %d - %s".formatted(mesa.getNumero(), mesa.getStatus().getDescricao()));
121         }
122     }
123
124     @FXML
125     private void handleAdicionarItem() {
126         if (mesaSelecionada == null) {
127             lblStatusBar.setText("Selecione uma mesa primeiro!");
128             return;
129         }
130         var u = CorubaFoodApp.getUsuarioLogado();
131         if (u == null) return;
132
133         try {
134             var produtos = produtoRepo.buscarTodos(u.getEmpresa().getId());

```



```

135         if (produtos.isEmpty()) {
136             lblStatusBar.setText("Nenhum produto cadastrado");
137             return;
138         }
139
140         var prod = produtos.get((int)(Math.random() * produtos.size()));
141         int qtd = (int)(Math.random() * 3) + 1;
142
143         var item = new ItemPedido();
144         item.setProduto(prod);
145         item.setQuantidade(qtd);
146         item.setPrecoUnitario(prod.getPreco());
147         item.setSubtotal(item.getPrecoUnitario().multiply(BigDecimal.valueOf(qtd)));
148
149         itensLista.add("%dx %s - R$ %s".formatted(qtd, prod.getNome(), item.getSubtotal()));
150         lblTotal.setText("R$ %s".formatted(item.getSubtotal().multiply(BigDecimal.valueOf(qtd))));
151         lblStatusBar.setText("%dx %s adicionado".formatted(qtd, prod.getNome()));
152     } catch (Exception e) {
153         logger.error("Erro ao adicionar item: {}", e.getMessage());
154         lblStatusBar.setText("Erro ao adicionar item");
155     }
156 }
157
158 @FXML
159 private void handleRemoverItem() {
160     var idx = lvItens.getSelectionModel().getSelectedIndex();
161     if (idx >= 0) {
162         itensLista.remove(idx);
163         lblStatusBar.setText("Item removido");
164     }
165 }
166
167 @FXML
168 private void handleFecharPedido() {
169     if (mesaSelecionada == null || itensLista.isEmpty()) {
170         lblStatusBar.setText("Nenhum item no pedido");
171         return;
172     }
173     lblStatusBar.setText("Pedido fechado - Total: %s".formatted(lblTotal.getText()));
174 }
175
176 @FXML
177 private void handlePagamento() {
178     if (mesaSelecionada == null) {
179         lblStatusBar.setText("Selecione uma mesa");
180         return;
181     }
182     lblStatusBar.setText("Pagamento registrado - %s".formatted(lblTotal.getText()));
183 }
184
185 @FXML
186 private void handleSair() {
187     try {
188         CorubaFoodApp.mostrarLogin();
189     } catch (Exception e) {
190         logger.error("Erro ao sair: {}", e.getMessage());
191     }
192 }
193
194 @FXML
195 private void handleDelivery() {
196     try {
197         CorubaFoodApp.mostrarDelivery();
198     } catch (Exception e) {
199         logger.error("Erro ao abrir delivery: {}", e.getMessage());
200         lblStatusBar.setText("Erro ao abrir Delivery: " + e.getMessage());
201     }
202 }
203 }
204

```

```
1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "categoria", uniqueConstraints = {
8     @UniqueConstraint(columnNames = {"nome", "empresa_id"}, name = "uk_categoria_nome_empresa")
9 })
10 public class Categoria {
11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
12     @Column(nullable = false, length = 100) private String nome;
13     @Column(length = 255) private String descricao;
14     @Column(name = "ordem") private Integer ordem = 0;
15     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
16     @Column(name = "ativo", nullable = false) private Boolean ativo = true;
17     @Column(name = "created_at") private LocalDateTime createdAt;
18     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
19     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
20     public String getNome() { return nome; } public void setNome(String n) { this.nome = n; }
21     public String getDescricao() { return descricao; } public void setDescricao(String d) { this.descricao = d; }
22     public Integer getOrdem() { return ordem; } public void setOrdem(Integer o) { this.ordem = o; }
23     public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
24     public Boolean getAtivo() { return ativo; } public void setAtivo(Boolean a) { this.ativo = a; }
25     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
26 }
27
```

```
1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "empresa", uniqueConstraints = {
8     @UniqueConstraint(columnNames = "documento", name = "uk_empresa_documento")
9 })
10 public class Empresa {
11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
12     @Column(nullable = false, length = 200) private String nome;
13     @Column(nullable = false, length = 18) private String documento;
14     @Column(length = 20) private String telefone;
15     @Column(length = 300) private String endereco;
16     @Column(length = 100) private String cidade;
17     @Column(length = 2) private String estado;
18     @Column(name = "ativo", nullable = false) private Boolean ativo = true;
19     @Column(name = "created_at") private LocalDateTime createdAt;
20     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
21     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
22     public String getNome() { return nome; } public void setNome(String nome) { this.nome = nome; }
23     public String getDocumento() { return documento; } public void setDocumento(String d) { this.documento = d; }
24     public String getTelefone() { return telefone; } public void setTelefone(String t) { this.telefone = t; }
25     public String getEndereco() { return endereco; } public void setEndereco(String e) { this.endereco = e; }
26     public String getCidade() { return cidade; } public void setCidade(String c) { this.cidade = c; }
27     public String getEstado() { return estado; } public void setEstado(String e) { this.estado = e; }
28     public Boolean getAtivo() { return ativo; } public void setAtivo(Boolean a) { this.ativo = a; }
29     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
30 }
31
```

```
1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5
6 @Entity
7 @Table(name = "item_pedido")
8 public class ItemPedido {
9     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
10    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "pedido_id", nullable = false) private Pedido pedido;
11    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "produto_id", nullable = false) private Produto produto;
12    @Column(nullable = false) private Integer quantidade;
13    @Column(nullable = false, precision = 10, scale = 2) private BigDecimal precoUnitario;
14    @Column(precision = 10, scale = 2) private BigDecimal subtotal;
15    @Column(length = 300) private String observacao;
16    @PrePersist @PreUpdate protected void calcularSubtotal() {
17        if (precoUnitario != null && quantidade != null) this.subtotal = precoUnitario.multiply(BigDecimal.valueOf(quantidade));
18    }
19    public Long getId() { return id; } public void setId(Long id) { this.id = id; }
20    public Pedido getPedido() { return pedido; } public void setPedido(Pedido p) { this.pedido = p; }
21    public Produto getProduto() { return produto; } public void setProduto(Produto p) { this.produto = p; }
22    public Integer getQuantidade() { return quantidade; } public void setQuantidade(Integer q) { this.quantidade = q; }
23    public BigDecimal getPrecoUnitario() { return precoUnitario; } public void setPrecoUnitario(BigDecimal p) { this.precoUnitario = p; }
24    public BigDecimal getSubtotal() { return subtotal; } public void setSubtotal(BigDecimal s) { this.subtotal = s; }
25    public String getObservacao() { return observacao; } public void setObservacao(String o) { this.observacao = o; }
26 }
27
```

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5
6 /**
7  * ItemPedidoDelivery - Item individual de um pedido de delivery.
8  *
9  * Cada item representa um produto do cardápio que foi
10  * pedido através do iFood ou 99Food.
11  */
12 @Entity
13 @Table(name = "itens_pedido_delivery")
14 public class ItemPedidoDelivery {
15
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private Long id;
19
20     // Nome do produto no momento do pedido (pode mudar no cardápio depois)
21     @Column(name = "nome_produto", nullable = false, length = 200)
22     private String nomeProduto;
23
24     // Quantidade pedida
25     @Column(nullable = false)
26     private Integer quantidade;
27
28     // Preço unitário no momento do pedido
29     @Column(name = "preco_unitario", nullable = false, precision = 10, scale = 2)
30     private BigDecimal precoUnitario;
31
32     // Subtotal = quantidade × preço unitário
33     @Column(nullable = false, precision = 10, scale = 2)
34     private BigDecimal subtotal;
35
36     // Observações específicas do item (ex: "sem cebola", "mal passado")
37     @Column(name = "observacoes", length = 300)
38     private String observacoes;
39
40     // Combo/adicionais
41     @Column(name = "adicionais", length = 500)
42     private String adicionais;
43
44     // Construtor padrão
45     public ItemPedidoDelivery() {}
46
47     // Construtor com campos obrigatórios
48     public ItemPedidoDelivery(String nomeProduto, Integer quantidade, BigDecimal precoUnitario) {
49         this.nomeProduto = nomeProduto;
50         this.quantidade = quantidade;
51         this.precoUnitario = precoUnitario;
52         calcularSubtotal();
53     }
54
55     /**
56      * Calcula o subtotal do item
57      */
58     public void calcularSubtotal() {
59         if (this.precoUnitario != null && this.quantidade != null) {
60             this.subtotal = this.precoUnitario.multiply(BigDecimal.valueOf(this.quantidade));
61         }
62     }
63
64     // --- Getters e Setters ---
65
66     public Long getId() { return id; }
67     public void setId(Long id) { this.id = id; }
68
69     public String getNomeProduto() { return nomeProduto; }
70     public void setNomeProduto(String nomeProduto) { this.nomeProduto = nomeProduto; }
71
72     public Integer getQuantidade() { return quantidade; }
73     public void setQuantidade(Integer quantidade) {
74         this.quantidade = quantidade;
75         calcularSubtotal();
76     }
77
78     public BigDecimal getPrecoUnitario() { return precoUnitario; }
79     public void setPrecoUnitario(BigDecimal precoUnitario) {
80         this.precoUnitario = precoUnitario;
81         calcularSubtotal();
82     }
83
84     public BigDecimal getSubtotal() { return subtotal; }
85     public void setSubtotal(BigDecimal subtotal) { this.subtotal = subtotal; }
86
87     public String getObservacoes() { return observacoes; }
88     public void setObservacoes(String observacoes) { this.observacoes = observacoes; }
89
90     public String getAdicionais() { return adicionais; }
91     public void setAdicionais(String adicionais) { this.adicionais = adicionais; }
92
93     @Override
94     public String toString() {
95         return quantidade + "x " + nomeProduto + " = R$ " + subtotal;
96     }
97 }
98

```

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 /**
7  * Mesa - Mesa do restaurante (JDK 21)
8  * Usa var, pattern matching e records
9  */
10 @Entity
11 @Table(name = "mesa", uniqueConstraints = {
12     @UniqueConstraint(columnNames = {"numero", "empresa_id"}, name = "uk_mesa_numero_empresa")
13 })
14 public class Mesa {
15
16     // Enum com pattern matching (JDK 21)
17     public enum Status {
18         LIVRE, OCUPADA, RESERVADA;
19
20         public String getDescricao() {
21             return switch (this) {
22                 case LIVRE -> "Disponível";
23                 case OCUPADA -> "Em uso";
24                 case RESERVADA -> "Reservada";
25             };
26         }
27
28         public String getCor() {
29             return switch (this) {
30                 case LIVRE -> "#4ec3a3";
31                 case OCUPADA -> "#e94560";
32                 case RESERVADA -> "#f77f00";
33             };
34         }
35     }
36
37     // Record para DTO de criação (JDK 21)
38     public record CriarMesa(Integer numero, Integer capacidade, Long empresaId) {}
39
40     // Record para resposta (JDK 21)
41     public record MesaDTO(Long id, Integer numero, Integer capacidade, Status status, String empresaNome) {}
42
43     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
44     private Long id;
45
46     @Column(nullable = false)
47     private Integer numero;
48
49     @Column(nullable = false)
50     private Integer capacidade;
51
52     @Enumerated(EnumType.STRING)
53     @Column(nullable = false, length = 20)
54     private Status status = Status.LIVRE;
55
56     @ManyToOne(fetch = FetchType.LAZY)
57     @JoinColumn(name = "empresa_id", nullable = false)
58     private Empresa empresa;
59
60     @Column(name = "created_at")
61     private LocalDateTime createdAt;
62
63     @PrePersist
64     protected void onCreate() {
65         createdAt = LocalDateTime.now();
66     }
67
68     // Construtor padrão (necessário para JPA)
69     public Mesa() {}
70
71     // Construtor com var (JDK 21)
72     public Mesa(Integer numero, Integer capacidade, Empresa empresa) {
73         var cap = capacidade != null ? capacidade : 4;
74         this.numero = numero;
75         this.capacidade = cap;
76         this.empresa = empresa;
77         this.status = Status.LIVRE;
78     }
79
80     // Factory method usando record
81     public static Mesa from(CriarMesa dto, Empresa empresa) {
82         return new Mesa(dto.numero(), dto.capacidade(), empresa);
83     }
84
85     // Converte para DTO usando var
86     public MesaDTO toDTO() {
87         var nomeEmpresa = empresa != null ? empresa.getNome() : "N/A";
88         return new MesaDTO(id, numero, capacidade, status, nomeEmpresa);
89     }
90
91     // Getters e Setters
92     public Long getId() { return id; }
93     public void setId(Long id) { this.id = id; }
94     public Integer getNumero() { return numero; }
95     public void setNumero(Integer numero) { this.numero = numero; }
96     public Integer getCapacidade() { return capacidade; }
97     public void setCapacidade(Integer capacidade) { this.capacidade = capacidade; }
98     public Status getStatus() { return status; }
99     public void setStatus(Status status) { this.status = status; }
100    public Empresa getEmpresa() { return empresa; }
101    public void setEmpresa(Empresa empresa) { this.empresa = empresa; }
102    public LocalDateTime getCreatedAt() { return createdAt; }
103    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }
104
105    @Override
106    public String toString() {
107        return "Mesa %d (%d lugares) - %s".formatted(numero, capacidade, status.getDescricao());
108    }
109 }
110

```

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "pagamento")
9 public class Pagamento {
10     public enum FormaPagamento { DINHEIRO, CARTAO_CREDITO, CARTAO_DEBITO, PIX, VALE_REFEICAO }
11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
12     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "pedido_id", nullable = false) private Pedido pedido;
13     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
14     @Enumerated(EnumType.STRING) @Column(nullable = false, length = 30) private FormaPagamento formaPagamento;
15     @Column(nullable = false, precision = 10, scale = 2) private BigDecimal valor;
16     @Column(precision = 10, scale = 2) private BigDecimal desconto = BigDecimal.ZERO;
17     @Column(name = "desconto_autorizado_por", length = 255) private String descontoAutorizadoPor;
18     @Column(name = "created_at") private LocalDateTime createdAt;
19     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
20     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
21     public Pedido getPedido() { return pedido; } public void setPedido(Pedido p) { this.pedido = p; }
22     public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
23     public FormaPagamento getFormaPagamento() { return formaPagamento; } public void setFormaPagamento(FormaPagamento f) { this.formaPagamento = f; }
24     public BigDecimal getValor() { return valor; } public void setValor(BigDecimal v) { this.valor = v; }
25     public BigDecimal getDesconto() { return desconto; } public void setDesconto(BigDecimal d) { this.desconto = d; }
26     public String getDescontoAutorizadoPor() { return descontoAutorizadoPor; } public void setDescontoAutorizadoPor(String d) { this.descontoAutorizadoPor = d; }
27     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
28 }
29

```

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5 import java.time.LocalDateTime;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 @Entity
10 @Table(name = "pedido")
11 public class Pedido {
12     public enum Status { ABERTO, EM_PREPARO, PRONTO, ENTREGUE, CANCELADO };
13     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
14     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "mesa_id", nullable = false) private Mesa mesa;
15     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "usuario_id", nullable = false) private Usuario usuario;
16     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
17     @OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL, orphanRemoval = true) private List<ItemPedido> itens = new ArrayList<>();
18     @Column(precision = 10, scale = 2) private BigDecimal total = BigDecimal.ZERO;
19     @Enumerated(EnumType.STRING) @Column(nullable = false, length = 20) private Status status = Status.ABERTO;
20     @Column(length = 500) private String observacao;
21     @Column(name = "created_at") private LocalDateTime createdAt;
22     @Column(name = "updated_at") private LocalDateTime updatedAt;
23     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); updatedAt = LocalDateTime.now(); }
24     @PreUpdate protected void onUpdate() { updatedAt = LocalDateTime.now(); }
25     public void calcularTotal() { this.total = itens.stream().map(ItemPedido::getSubtotal).reduce(BigDecimal.ZERO, BigDecimal::add); }
26     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
27     public Mesa getMesa() { return mesa; } public void setMesa(Mesa m) { this.mesa = m; }
28     public Usuario getUsuario() { return usuario; } public void setUsuario(Usuario u) { this.usuario = u; }
29     public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
30     public List<ItemPedido> getItens() { return itens; } public void setItens(List<ItemPedido> i) { this.itens = i; }
31     public BigDecimal getTotal() { return total; } public void setTotal(BigDecimal t) { this.total = t; }
32     public Status getStatus() { return status; } public void setStatus(Status s) { this.status = s; }
33     public String getObservacao() { return observacao; } public void setObservacao(String o) { this.observacao = o; }
34     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
35     public LocalDateTime getUpdatedAt() { return updatedAt; } public void setUpdatedAt(LocalDateTime u) { this.updatedAt = u; }
36 }
37

```



```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5 import java.time.LocalDateTime;
6 import java.util.ArrayList;
7 import java.util.List;
8
9 /**
10  * PedidoDelivery - Pedidos vindos de plataformas de delivery (iFood, 99Food).
11  */
12 @Entity
13 @Table(name = "pedidos_delivery")
14 public class PedidoDelivery {
15
16     public enum Plataforma { IF00D, F00D99 }
17     public enum StatusPedido { NOVO, CONFIRMADO, PREPARANDO, PRONTO, SAIU_ENTREGA, ENTREGUE, CANCELADO }
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private Long id;
22
23     @Column(name = "codigo_plataforma", nullable = false, length = 50)
24     private String codigoPlataforma;
25
26     @Enumerated(EnumType.STRING)
27     @Column(nullable = false, length = 20)
28     private Plataforma plataforma;
29
30     @Column(name = "nome_cliente", nullable = false, length = 200)
31     private String nomeCliente;
32
33     @Column(name = "telefone_cliente", length = 20)
34     private String telefoneCliente;
35
36     @Column(name = "endereco_entrega", length = 500)
37     private String enderecoEntrega;
38
39     @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
40     @JoinColumn(name = "pedido_delivery_id")
41     private List<ItemPedidoDelivery> itens = new ArrayList<>();
42
43     @Column(name = "valor_itens", nullable = false, precision = 10, scale = 2)
44     private BigDecimal valorItens;
45
46     @Column(name = "taxa_entrega", precision = 10, scale = 2)
47     private BigDecimal taxaEntrega = BigDecimal.ZERO;
48
49     @Column(name = "valor_total", nullable = false, precision = 10, scale = 2)
50     private BigDecimal valorTotal;
51
52     @Enumerated(EnumType.STRING)
53     @Column(nullable = false, length = 20)
54     private StatusPedido status = StatusPedido.NOVO;
55
56     @Column(name = "forma_pagamento", length = 50)
57     private String formaPagamento;
58
59     @Column(name = "observacoes", length = 500)
60     private String observacoes;
61
62     @ManyToOne(fetch = FetchType.LAZY)
63     @JoinColumn(name = "empresa_id", nullable = false)
64     private Empresa empresa;
65
66     @Column(name = "data_criacao", nullable = false)
67     private LocalDateTime dataCriacao = LocalDateTime.now();
68
69     @Column(name = "data_confirmacao")
70     private LocalDateTime dataConfirmacao;
71
72     @Column(name = "data_pronto")
73     private LocalDateTime dataPronto;
74
75     @Column(name = "data_entrega")
76     private LocalDateTime dataEntrega;
77
78     @Column(name = "data_cancelamento")
79     private LocalDateTime dataCancelamento;
80
81     @Column(name = "motivo_cancelamento", length = 500)
82     private String motivoCancelamento;
83
84     public PedidoDelivery() {}
85
86     public PedidoDelivery(Plataforma plataforma, String codigoPlataforma, String nomeCliente, Empresa empresa) {
87         this.plataforma = plataforma;
88         this.codigoPlataforma = codigoPlataforma;
89         this.nomeCliente = nomeCliente;
90         this.empresa = empresa;
91         this.valorItens = BigDecimal.ZERO;
92         this.valorTotal = BigDecimal.ZERO;
93         this.status = StatusPedido.NOVO;
94     }
95
96     public void calcularTotal() {
97         if (itens == null || itens.isEmpty()) {
98             this.valorItens = BigDecimal.ZERO;
99         } else {
100             this.valorItens = itens.stream()
101                 .map(ItemPedidoDelivery::getSubtotal)
102                 .reduce(BigDecimal.ZERO, BigDecimal::add);
103         }
104         BigDecimal taxa = this.taxaEntrega != null ? this.taxaEntrega : BigDecimal.ZERO;
105         this.valorTotal = this.valorItens.add(taxa);
106     }
107
108     public boolean avancarStatus() {
109         switch (this.status) {
110             case NOVO: this.status = StatusPedido.CONFIRMADO; this.dataConfirmacao = LocalDateTime.now(); return true;
111             case CONFIRMADO: this.status = StatusPedido.PREPARANDO; return true;
112             case PREPARANDO: this.status = StatusPedido.PRONTO; this.dataPronto = LocalDateTime.now(); return true;
113             case PRONTO: this.status = StatusPedido.SAIU_ENTREGA; return true;
114             case SAIU_ENTREGA: this.status = StatusPedido.ENTREGUE; this.dataEntrega = LocalDateTime.now(); return true;
115             default: return false;
116         }
117     }
118
119     public void cancelar(String motivo) {
120         this.status = StatusPedido.CANCELADO;
121         this.motivoCancelamento = motivo;
122         this.dataCancelamento = LocalDateTime.now();
123     }
124
125     // Getters e Setters
126     public Long getId() { return id; }
127     public void setId(Long id) { this.id = id; }
128     public String getCodigoPlataforma() { return codigoPlataforma; }
129     public void setCodigoPlataforma(String c) { this.codigoPlataforma = c; }
130     public Plataforma getPlataforma() { return plataforma; }
131     public void setPlataforma(Plataforma p) { this.plataforma = p; }
132     public String getNomeCliente() { return nomeCliente; }
133     public void setNomeCliente(String n) { this.nomeCliente = n; }
134     public String getTelefoneCliente() { return telefoneCliente; }

```

```

135 public void setTelefoneCliente(String t) { this.telefoneCliente = t; }
136 public String getEnderecoEntrega() { return enderecoEntrega; }
137 public void setEnderecoEntrega(String e) { this.enderecoEntrega = e; }
138 public List<ItemPedidoDelivery> getItens() { return itens; }
139 public void setItens(List<ItemPedidoDelivery> i) { this.itens = i; }
140 public BigDecimal getValorItens() { return valorItens; }
141 public void setValorItens(BigDecimal v) { this.valorItens = v; }
142 public BigDecimal getTaxaEntrega() { return taxaEntrega; }
143 public void setTaxaEntrega(BigDecimal t) { this.taxaEntrega = t; }
144 public BigDecimal getValorTotal() { return valorTotal; }
145 public void setValorTotal(BigDecimal v) { this.valorTotal = v; }
146 public StatusPedido getStatus() { return status; }
147 public void setStatus(StatusPedido s) { this.status = s; }
148 public String getFormaPagamento() { return formaPagamento; }
149 public void setFormaPagamento(String f) { this.formaPagamento = f; }
150 public String getObservacoes() { return observacoes; }
151 public void setObservacoes(String o) { this.observacoes = o; }
152 public Empresa getEmpresa() { return empresa; }
153 public void setEmpresa(Empresa e) { this.empresa = e; }
154 public LocalDateTime getDataCriacao() { return dataCriacao; }
155 public void setDataCriacao(LocalDateTime d) { this.dataCriacao = d; }
156 public LocalDateTime getDataConfirmacao() { return dataConfirmacao; }
157 public void setDataConfirmacao(LocalDateTime d) { this.dataConfirmacao = d; }
158 public LocalDateTime getDataPronto() { return dataPronto; }
159 public void setDataPronto(LocalDateTime d) { this.dataPronto = d; }
160 public LocalDateTime getDataEntrega() { return dataEntrega; }
161 public void setDataEntrega(LocalDateTime d) { this.dataEntrega = d; }
162 public LocalDateTime getDataCancelamento() { return dataCancelamento; }
163 public void setDataCancelamento(LocalDateTime d) { this.dataCancelamento = d; }
164 public String getMotivoCancelamento() { return motivoCancelamento; }
165 public void setMotivoCancelamento(String m) { this.motivoCancelamento = m; }
166 }
167

```

```

1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5 import java.time.LocalDateTime;
6
7 @Entity
8 @Table(name = "produto", uniqueConstraints = {
9     @UniqueConstraint(columnNames = {"codigo", "empresa_id"}, name = "uk_produto_codigo_empresa")
10 })
11 public class Produto {
12     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
13     @Column(nullable = false, length = 50) private String codigo;
14     @Column(nullable = false, length = 200) private String nome;
15     @Column(length = 500) private String descricao;
16     @Column(nullable = false, precision = 10, scale = 2) private BigDecimal preco;
17     @Column(name = "imagem_path", length = 500) private String imagePath;
18     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "categoria_id") private Categoria categoria;
19     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
20     @Column(name = "disponivel", nullable = false) private Boolean disponivel = true;
21     @Column(name = "ativo", nullable = false) private Boolean ativo = true;
22     @Column(name = "created_at") private LocalDateTime createdAt;
23     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
24     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
25     public String getCodigo() { return codigo; } public void setCodigo(String c) { this.codigo = c; }
26     public String getNome() { return nome; } public void setNome(String n) { this.nome = n; }
27     public String getDescricao() { return descricao; } public void setDescricao(String d) { this.descricao = d; }
28     public BigDecimal getPreco() { return preco; } public void setPreco(BigDecimal p) { this.preco = p; }
29     public String getImagePath() { return imagePath; } public void setImagePath(String i) { this.imagePath = i; }
30     public Categoria getCategoria() { return categoria; } public void setCategoria(Categoria c) { this.categoria = c; }
31     public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
32     public Boolean getDisponivel() { return disponivel; } public void setDisponivel(Boolean d) { this.disponivel = d; }
33     public Boolean getAtivo() { return ativo; } public void setAtivo(Boolean a) { this.ativo = a; }
34     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
35 }
36

```

```
1 package com.corumbasistemas.corubafood.model;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "usuario", uniqueConstraints = {
8     @UniqueConstraint(columnNames = {"username", "empresa_id"}, name = "uk_usuario_username_empresa")
9 })
10 public class Usuario {
11     public enum Papel { ADMIN, GARCOM, COZINHA, CAIXA }
12     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
13     @Column(nullable = false, length = 100) private String nome;
14     @Column(nullable = false, length = 100) private String username;
15     @Column(nullable = false, length = 255) private String senha;
16     @Enumerated(EnumType.STRING) @Column(nullable = false, length = 20) private Papel papel;
17     @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "empresa_id", nullable = false) private Empresa empresa;
18     @Column(name = "ativo", nullable = false) private Boolean ativo = true;
19     @Column(name = "created_at") private LocalDateTime createdAt;
20     @PrePersist protected void onCreate() { createdAt = LocalDateTime.now(); }
21     public Long getId() { return id; } public void setId(Long id) { this.id = id; }
22     public String getNome() { return nome; } public void setNome(String n) { this.nome = n; }
23     public String getUsername() { return username; } public void setUsername(String u) { this.username = u; }
24     public String getSenha() { return senha; } public void setSenha(String s) { this.senha = s; }
25     public Papel getPapel() { return papel; } public void setPapel(Papel p) { this.papel = p; }
26     public Empresa getEmpresa() { return empresa; } public void setEmpresa(Empresa e) { this.empresa = e; }
27     public Boolean getAtivo() { return ativo; } public void setAtivo(Boolean a) { this.ativo = a; }
28     public LocalDateTime getCreatedAt() { return createdAt; } public void setCreatedAt(LocalDateTime c) { this.createdAt = c; }
29 }
30
```

```
1 package com.corumbasistemas.corubafood.repository;
2 import com.corumbasistemas.corubafood.model.Pagamento;
3 import com.corumbasistemas.corubafood.util.JPAUtil;
4 import jakarta.persistence.*;
5 import java.util.List;
6 public class PagamentoRepository {
7     public Pagamento buscarPorPedido(Long eid, Long pid) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT p FROM Pagamento p WHERE p.empresa.id=:eid AND
p.pedido.id=:pid", Pagamento.class).setParameter("eid", eid).setParameter("pid", pid).getSingleResult(); } catch (NoResultException e) { return null; } finally { em.close(); } }
8     public List<Pagamento> buscarTodos(Long eid) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT p FROM Pagamento p WHERE p.empresa.id=:eid ORDER BY
p.createdAt DESC", Pagamento.class).setParameter("eid", eid).getResultList(); } finally { em.close(); } }
9     public Pagamento salvar(Pagamento p) { EntityManager em = JPAUtil.getEM(); try { em.getTransaction().begin(); if (p.getId() == null) em.persist(p); else p = em.merge(p);
em.getTransaction().commit(); return p; } catch (Exception e) { if (em.getTransaction().isActive()) em.getTransaction().rollback(); throw e; } finally { em.close(); } }
10 }
```

```
1 package com.corumbasistemas.corubafood.repository;
2 import com.corumbasistemas.corubafood.model.*;
3 import com.corumbasistemas.corubafood.util.JPAUtil;
4 import jakarta.persistence.*;
5 import java.util.List;
6 public class ProdutoRepository {
7     public List<Produto> buscarTodos(Long eid) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT p FROM Produto p WHERE p.empresa.id=:eid AND p.ativo=true
ORDER BY p.categoria.nome, p.nome", Produto.class).setParameter("eid", eid).getResultList(); } finally { em.close(); } }
8     public List<Produto> buscarPorCategoria(Long eid, Long cid) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT p FROM Produto p WHERE p.empresa.id=:eid AND
p.categoria.id=:cid AND p.ativo=true ORDER BY p.nome", Produto.class).setParameter("eid", eid).setParameter("cid", cid).getResultList(); } finally { em.close(); } }
9     public Produto salvar(Produto p) { EntityManager em = JPAUtil.getEM(); try { em.getTransaction().begin(); if (p.getId() == null) em.persist(p); else p = em.merge(p);
em.getTransaction().commit(); return p; } catch (Exception e) { if (em.getTransaction().isActive()) em.getTransaction().rollback(); throw e; } finally { em.close(); } }
10    public List<Categoria> buscarCategorias(Long eid) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT c FROM Categoria c WHERE c.empresa.id=:eid AND
c.ativo=true ORDER BY c.ordem, c.nome", Categoria.class).setParameter("eid", eid).getResultList(); } finally { em.close(); } }
11 }
```

```
1 package com.corumbasistemas.corubafood.repository;
2 import com.corumbasistemas.corubafood.model.Usuario;
3 import com.corumbasistemas.corubafood.util.JPAUtil;
4 import jakarta.persistence.*;
5 import java.util.List;
6 public class UsuarioRepository {
7     public Usuario buscarPorUsername(Long eid, String u) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT u FROM Usuario u WHERE u.empresa.id=:eid AND
u.username=:u AND u.ativo=true", Usuario.class).setParameter("eid", eid).setParameter("u", u).getSingleResult(); } catch (NoResultException e) { return null; } finally { em.close(); } }
8     public List<Usuario> buscarTodos(Long eid) { EntityManager em = JPAUtil.getEM(); try { return em.createQuery("SELECT u FROM Usuario u WHERE u.empresa.id=:eid AND u.ativo=true
ORDER BY u.nome", Usuario.class).setParameter("eid", eid).getResultList(); } finally { em.close(); } }
9     public Usuario salvar(Usuario u) { EntityManager em = JPAUtil.getEM(); try { em.getTransaction().begin(); if (u.getId() == null) em.persist(u); else u = em.merge(u);
em.getTransaction().commit(); return u; } catch (Exception e) { if (em.getTransaction().isActive()) em.getTransaction().rollback(); throw e; } finally { em.close(); } }
10 }
```

```
1 package com.corumbasistemas.corubafood.security;
2 import org.mindrot.jbcrypt.BCrypt;
3 import org.slf4j.*;
4 public class PasswordHash {
5     private static final Logger logger = LoggerFactory.getLogger(PasswordHash.class);
6     private static final int WORKLOAD = 12;
7     private PasswordHash() {}
8     public static String hash(String s) {
9         if (s == null || s.isEmpty()) throw new IllegalArgumentException();
10        return BCrypt.hashpw(s, BCrypt.gensalt(WORKLOAD));
11    }
12    public static boolean verify(String s, String stored) {
13        if (s == null || stored == null) return false;
14        if (isBCrypt(stored)) { try { return BCrypt.checkpw(s, stored); } catch (Exception e) { return false; } }
15        logger.info("Plaintext detected - needs BCrypt migration");
16        return s.equals(stored);
17    }
18    public static boolean isBCrypt(String v) { return v != null && v.startsWith("$2a$") && v.length() == 60; }
19    public static boolean needsRehash(String s) { return !isBCrypt(s) || !s.startsWith("$2a$" + WORKLOAD + "$"); }
20 }
```



```

1 package com.corumbasistemas.corubafood.test;
2
3 import com.corumbasistemas.corubafood.controller.AuthController;
4 import com.corumbasistemas.corubafood.model.*;
5 import com.corumbasistemas.corubafood.repository.*;
6 import com.corumbasistemas.corubafood.security.PasswordHash;
7 import com.corumbasistemas.corubafood.util.*;
8 import jakarta.persistence.EntityManager;
9 import org.slf4j.*;
10
11 import java.math.BigDecimal;
12 import java.util.List;
13
14 /**
15  * Teste completo do MesaPronta via JPA (sem GUI).
16  * Valida: seed massivo, login, CRUD, pedidos, pagamentos.
17  */
18 public class TesteMesaProntaCompleto {
19     private static final Logger logger = LoggerFactory.getLogger(TesteMesaProntaCompleto.class);
20     private static int testesPassaram = 0;
21     private static int testesFalharam = 0;
22     private static final StringBuilder relatorio = new StringBuilder();
23
24     public static void main(String[] args) {
25         long inicio = System.currentTimeMillis();
26         relatorio.append("===== \n");
27         relatorio.append("  TESTE COMPLETO MESAPRONTA v2.0 \n");
28         relatorio.append("  Corumbá Sistemas \n");
29         relatorio.append("===== \n \n");
30
31         try {
32             // Inicializa JPA
33             logger.info("Inicializando JPA...");
34             JPAUtil.getEMF();
35
36             // 1. Seed normal (demo)
37             testar("Seed Demo", () -> {
38                 SeedData.popular();
39                 return true;
40             });
41
42             // 2. Seed Massivo (100 empresas)
43             testar("Seed Massivo (100 empresas)", () -> {
44                 SeedMassivo.popular();
45                 return true;
46             });
47
48             // 3. Contagem de registros
49             testar("Contagem: 100+ empresas", () -> {
50                 EntityManager em = JPAUtil.getEM();
51                 Long count = em.createQuery("SELECT COUNT(e) FROM Empresa e", Long.class).getSingleResult();
52                 em.close();
53                 return count >= 100;
54             });
55
56             testar("Contagem: 1000+ usuários", () -> {
57                 EntityManager em = JPAUtil.getEM();
58                 Long count = em.createQuery("SELECT COUNT(u) FROM Usuario u", Long.class).getSingleResult();
59                 em.close();
60                 return count >= 1000;
61             });
62
63             testar("Contagem: 10000+ produtos", () -> {
64                 EntityManager em = JPAUtil.getEM();
65                 Long count = em.createQuery("SELECT COUNT(p) FROM Produto p", Long.class).getSingleResult();
66                 em.close();
67                 return count >= 10000;
68             });
69
70             testar("Contagem: 1000+ mesas", () -> {
71                 EntityManager em = JPAUtil.getEM();
72                 Long count = em.createQuery("SELECT COUNT(m) FROM Mesa m", Long.class).getSingleResult();
73                 em.close();
74                 return count >= 1000;
75             });
76
77             testar("Contagem: 500+ categorias", () -> {
78                 EntityManager em = JPAUtil.getEM();
79                 Long count = em.createQuery("SELECT COUNT(c) FROM Categoria c", Long.class).getSingleResult();
80                 em.close();
81                 return count >= 500;
82             });
83
84             // 4. Teste de login
85             testar("Login admin/demo", () -> {
86                 AuthController auth = new AuthController();
87                 Empresa emp = auth.buscarEmpresaPorDocumento("12.345.678/0001-90");
88                 if (emp == null) return false;
89                 Usuario u = auth.autenticar(emp.getId(), "admin", "admin123");
90                 return u != null && u.getPapel() == Usuario.Papel.ADMIN;
91             });
92
93             // 5. Teste de login com senha errada
94             testar("Login senha errada rejeita", () -> {
95                 AuthController auth = new AuthController();
96                 Empresa emp = auth.buscarEmpresaPorDocumento("12.345.678/0001-90");
97                 if (emp == null) return false;
98                 Usuario u = auth.autenticar(emp.getId(), "admin", "senhaerrada");
99                 return u == null;
100             });
101
102             // 6. Teste de login garcom
103             testar("Login garcom/demo", () -> {
104                 AuthController auth = new AuthController();
105                 Empresa emp = auth.buscarEmpresaPorDocumento("12.345.678/0001-90");
106                 if (emp == null) return false;
107                 Usuario u = auth.autenticar(emp.getId(), "garcom", "garcom123");
108                 return u != null && u.getPapel() == Usuario.Papel.GARCOM;
109             });
110
111             // 7. CRUD Produtos
112             testar("CRUD Produto - buscar todos", () -> {
113                 ProdutoRepository repo = new ProdutoRepository();
114                 Empresa emp = new AuthController().buscarEmpresaPorDocumento("12.345.678/0001-90");
115                 if (emp == null) return false;
116                 List<Produto> prods = repo.buscarTodos(emp.getId());
117                 return prods != null && prods.size() >= 5; // seed demo tem 5
118             });
119
120             testar("CRUD Produto - buscar por categoria", () -> {
121                 ProdutoRepository repo = new ProdutoRepository();
122                 Empresa emp = new AuthController().buscarEmpresaPorDocumento("12.345.678/0001-90");
123                 if (emp == null) return false;
124                 List<Categoria> cats = repo.buscarCategorias(emp.getId());
125                 if (cats.isEmpty()) return false;
126                 List<Produto> prods = repo.buscarPorCategoria(emp.getId(), cats.get(0).getId());
127                 return prods != null && !prods.isEmpty();
128             });
129
130             // 8. CRUD Usuários
131             testar("CRUD Usuario - buscar todos", () -> {
132                 UsuarioRepository repo = new UsuarioRepository();
133                 Empresa emp = new AuthController().buscarEmpresaPorDocumento("12.345.678/0001-90");
134                 if (emp == null) return false;

```

```

135         List<Usuario> users = repo.buscarTodos(emp.getId());
136         return users != null && users.size() >= 2; // admin + garcom
137     });
138
139     // 9. Criar pedido
140     testar("Criar pedido completo", () -> {
141         EntityManager em = JPAUtil.getEM();
142         try {
143             Empresa emp = em.createQuery("SELECT e FROM Empresa e WHERE e.documento='12.345.678/0001-90'", Empresa.class)
144                 .getSingleResult();
145             Usuario user = em.createQuery("SELECT u FROM Usuario u WHERE u.username='admin'", Usuario.class)
146                 .getSingleResult();
147             Mesa mesa = em.createQuery("SELECT m FROM Mesa m WHERE m.numero=1 AND m.empresa.id=:eid", Mesa.class)
148                 .setParameter("eid", emp.getId()).getSingleResult();
149             Produto prod = em.createQuery("SELECT p FROM Produto p WHERE p.empresa.id=:eid", Produto.class)
150                 .setParameter("eid", emp.getId()).setMaxResults(1).getSingleResult();
151
152             em.getTransaction().begin();
153
154             Pedido pedido = new Pedido();
155             pedido.setMesa(mesa);
156             pedido.setUsuario(user);
157             pedido.setEmpresa(emp);
158             pedido.setStatus(Pedido.Status.ABERTO);
159             em.persist(pedido);
160             em.flush();
161
162             ItemPedido item = new ItemPedido();
163             item.setPedido(pedido);
164             item.setProduto(prod);
165             item.setQuantidade(2);
166             item.setPrecoUnitario(prod.getPreco());
167             item.setSubtotal(item.getPrecoUnitario().multiply(BigDecimal.valueOf(item.getQuantidade())));
168             em.persist(item);
169
170             mesa.setStatus(Mesa.Status.OCUPADA);
171
172             em.getTransaction().commit();
173
174             // Verificar
175             return pedido.getId() != null && item.getId() != null;
176         } catch (Exception e) {
177             if (em.getTransaction().isActive()) em.getTransaction().rollback();
178             logger.error("Erro: {}", e.getMessage());
179             return false;
180         } finally {
181             em.close();
182         }
183     });
184
185     // 10. Criar pagamento
186     testar("Criar pagamento", () -> {
187         EntityManager em = JPAUtil.getEM();
188         try {
189             Pedido pedido = em.createQuery("SELECT p FROM Pedido p", Pedido.class)
190                 .setMaxResults(1).getSingleResult();
191
192             em.getTransaction().begin();
193
194             Pagamento pag = new Pagamento();
195             pag.setPedido(pedido);
196             pag.setEmpresa(pedido.getEmpresa());
197             pag.setFormaPagamento(Pagamento.FormaPagamento.PIX);
198             pag.setValor(new BigDecimal("91.80"));
199             em.persist(pag);
200
201             em.getTransaction().commit();
202             return pag.getId() != null;
203         } catch (Exception e) {
204             if (em.getTransaction().isActive()) em.getTransaction().rollback();
205             logger.error("Erro: {}", e.getMessage());
206             return false;
207         } finally {
208             em.close();
209         }
210     });
211
212     // 11. BCrypt - hash e verify
213     testar("BCrypt - hash e verify", () -> {
214         String hash = PasswordHash.hash("teste123");
215         return PasswordHash.verify("teste123", hash) && !PasswordHash.verify("errado", hash);
216     });
217
218     // 12. BCrypt - migração de plaintext
219     testar("BCrypt - migração plaintext", () -> {
220         String plaintext = "senha123";
221         return PasswordHash.verify("senha123", plaintext) && PasswordHash.needsRehash(plaintext);
222     });
223
224     // 13. Busca de empresa por documento
225     testar("Buscar empresa por CNPJ", () -> {
226         AuthController auth = new AuthController();
227         Empresa e = auth.buscarEmpresaPorDocumento("12.345.678/0001-90");
228         return e != null && e.getNome().contains("Demo");
229     });
230
231     // 14. Empresa inexistente
232     testar("Empresa inexistente retorna null", () -> {
233         AuthController auth = new AuthController();
234         Empresa e = auth.buscarEmpresaPorDocumento("00.000.000/0000-00");
235         return e == null;
236     });
237
238     // 15. IsAdmin check
239     testar("Verificar isAdmin", () -> {
240         AuthController auth = new AuthController();
241         Empresa emp = auth.buscarEmpresaPorDocumento("12.345.678/0001-90");
242         if (emp == null) return false;
243         Usuario admin = auth.autenticar(emp.getId(), "admin", "admin123");
244         Usuario garcom = auth.autenticar(emp.getId(), "garcom", "garcom123");
245         return auth.isAdmin(admin) && !auth.isAdmin(garcom);
246     });
247
248     // 16. Performance: consulta 100 empresas
249     testar("Performance: listar 100 empresas < 1s", () -> {
250         long t0 = System.currentTimeMillis();
251         EntityManager em = JPAUtil.getEM();
252         List<Empresa> empresas = em.createQuery("SELECT e FROM Empresa e ORDER BY e.nome", Empresa.class)
253             .getResultList();
254         em.close();
255         long elapsed = System.currentTimeMillis() - t0;
256         logger.info(" → {} empresas consultadas em {}ms", empresas.size(), elapsed);
257         return empresas.size() >= 100 && elapsed < 1000;
258     });
259
260     // 17. Performance: consulta 10000 produtos
261     testar("Performance: contar 10000+ produtos < 1s", () -> {
262         long t0 = System.currentTimeMillis();
263         EntityManager em = JPAUtil.getEM();
264         Long count = em.createQuery("SELECT COUNT(p) FROM Produto p", Long.class).getSingleResult();
265         em.close();
266         long elapsed = System.currentTimeMillis() - t0;
267         logger.info(" → {} produtos contados em {}ms", count, elapsed);
268         return count >= 10000 && elapsed < 1000;
269     });
270
271     // 18. Integridade: empresa tem usuários
272     testar("Integridade: empresa tem 10+ usuários", () -> {

```

```

EntityManager em = JPAUtil.getEM();
List<Empresa> empresas = em.createQuery("SELECT e FROM Empresa e", Empresa.class)
    .setMaxResults(10).getResultList();
for (Empresa e : empresas) {
    Long count = em.createQuery(
        "SELECT COUNT(u) FROM Usuario u WHERE u.empresa.id=:eid", Long.class)
        .setParameter("eid", e.getId()).getSingleResult();
    if (count < 2) { em.close(); return false; }
}
em.close();
return true;
});

// 19. Integridade: empresa tem produtos
testar("Integridade: empresa tem 5+ produtos", () -> {
    EntityManager em = JPAUtil.getEM();
    List<Empresa> empresas = em.createQuery("SELECT e FROM Empresa e", Empresa.class)
        .setMaxResults(10).getResultList();
    for (Empresa e : empresas) {
        Long count = em.createQuery(
            "SELECT COUNT(p) FROM Produto p WHERE p.empresa.id=:eid", Long.class)
            .setParameter("eid", e.getId()).getSingleResult();
        if (count < 5) { em.close(); return false; }
    }
    em.close();
    return true;
});

// 20. Integridade: empresa tem mesas
testar("Integridade: empresa tem 10 mesas", () -> {
    EntityManager em = JPAUtil.getEM();
    List<Empresa> empresas = em.createQuery("SELECT e FROM Empresa e", Empresa.class)
        .setMaxResults(10).getResultList();
    for (Empresa e : empresas) {
        Long count = em.createQuery(
            "SELECT COUNT(m) FROM Mesa m WHERE m.empresa.id=:eid", Long.class)
            .setParameter("eid", e.getId()).getSingleResult();
        if (count != 10) { em.close(); return false; }
    }
    em.close();
    return true;
});

} catch (Exception e) {
    logger.error("Erro fatal: {}", e.getMessage(), e);
    relatorio.append("\nX ERRO FATAL: ").append(e.getMessage()).append("\n");
} finally {
    JPAUtil.shutdown();
}

long fim = System.currentTimeMillis();
long total = fim - inicio;

// Relatório final
relatorio.append("\n===== \n");
relatorio.append(" RESULTADO FINAL \n");
relatorio.append("===== \n");
relatorio.append(String.format(" ✓ Passaram: %d \n", testesPassaram));
relatorio.append(String.format(" ✗ Falharam: %d \n", testesFalharam));
relatorio.append(String.format(" Total: %d \n", testesPassaram + testesFalharam));
relatorio.append(String.format(" ⌚ Tempo: %dms (%.1fs) \n", total, total / 1000.0));
relatorio.append("===== \n");

if (testesFalharam == 0) {
    relatorio.append("\n🎉 TODOS OS TESTES PASSARAM! 🎉 \n");
} else {
    relatorio.append(String.format("\n🚫 %d teste(s) falharam. Verifique o log. \n", testesFalharam));
}

String resultado = relatorio.toString();
logger.info("\n{}", resultado);

// Salvar relatório em arquivo
try {
    java.nio.file.Files.writeString(
        java.nio.file.Path.of("/opt/projects/coruba-food/relatorio_testes.txt"),
        resultado);
    logger.info("Relatório salvo em /opt/projects/coruba-food/relatorio_testes.txt");
} catch (Exception e) {
    logger.warn("Não foi possível salvar relatório: {}", e.getMessage());
}

System.exit(testesFalharam > 0 ? 1 : 0);
}

@FunctionalInterface
interface TesteAction {
    boolean execute() throws Exception;
}

private static void testar(String nome, TesteAction action) {
    try {
        long t0 = System.currentTimeMillis();
        boolean resultado = action.execute();
        long elapsed = System.currentTimeMillis() - t0;

        if (resultado) {
            testesPassaram++;
            logger.info(" ✓ PASSOU [{}ms] - {}", elapsed, nome);
            relatorio.append(String.format(" ✓ PASSOU [{}dms] %s \n", elapsed, nome));
        } else {
            testesFalharam++;
            logger.error(" ✗ FALHOU [{}ms] - {}", elapsed, nome);
            relatorio.append(String.format(" ✗ FALHOU [{}dms] %s \n", elapsed, nome));
        }
    } catch (Exception e) {
        testesFalharam++;
        logger.error(" ✗ ERRO - {}: {}", nome, e.getMessage());
        relatorio.append(String.format(" ✗ ERRO - %s: %s \n", nome, e.getMessage()));
    }
}
}
}

```

```
1 package com.corumbasistemas.corubafood.util;
2 import jakarta.persistence.*;
3 import org.slf4j.*;
4 import java.util.*;
5 public class JPAUtil {
6     private static final Logger logger = LoggerFactory.getLogger(JPAUtil.class);
7     private static volatile EntityManagerFactory emf;
8     private static final Object lock = new Object();
9     private JPAUtil() {}
10    public static EntityManagerFactory getEMF() {
11        if (emf == null) { synchronized (lock) {
12            if (emf == null) { try {
13                Map<String, String> p = new HashMap<>();
14                String db = System.getenv("DB_PATH");
15                if (db != null && !db.isEmpty()) p.put("jakarta.persistence.jdbc.url", "jdbc:sqlite:" + db);
16                String h = System.getenv("HBM2DDL_AUTO");
17                if (h != null) p.put("hibernate.hbm2ddl.auto", h);
18                emf = Persistence.createEntityManagerFactory("coruba-food-pu", p);
19                logger.info("EMF created");
20                Runtime.getRuntime().addShutdownHook(new Thread(() -> { if (emf != null && emf.isOpen()) { emf.close(); logger.info("EMF closed"); } }));
21            } catch (Exception e) { logger.error("EMF error: {}", e.getMessage(), e); throw new RuntimeException("JPA init failed", e); }
22            }}}
23        return emf;
24    }
25    public static EntityManager getEM() { return getEMF().createEntityManager(); }
26    public static void shutdown() { synchronized (lock) { if (emf != null && emf.isOpen()) { emf.close(); emf = null; } } }
27 }
```

```

1 package com.corumbasistemas.corubafood.util;
2 import com.corumbasistemas.corubafood.model.*;
3 import com.corumbasistemas.corubafood.security.PasswordHash;
4 import jakarta.persistence.EntityManager;
5 import org.slf4j.*;
6 import java.math.BigDecimal;
7 public class SeedData {
8     private static final Logger logger = LoggerFactory.getLogger(SeedData.class);
9     public static void popular() {
10         EntityManager em = JPAUtil.getEM();
11         try {
12             em.getTransaction().begin();
13             Long c = em.createQuery("SELECT COUNT(e) FROM Empresa e", Long.class).getSingleResult();
14             if (c > 0) { logger.info("DB already seeded"); em.getTransaction().commit(); return; }
15             Empresa e = new Empresa(); e.setNome("Restaurante Demo"); e.setDocumento("12.345.678/0001-90"); e.setTelefone("(11) 99999-0000"); e.setEndereco("Rua da Comida, 100");
16             e.setCidade("Corumbá"); e.setEstado("MS"); em.persist(e); em.flush();
17             Categoria c1 = new Categoria(); c1.setNome("Pratos Principais"); c1.setOrdem(1); c1.setEmpresa(e); em.persist(c1);
18             Categoria c2 = new Categoria(); c2.setNome("Bebidas"); c2.setOrdem(2); c2.setEmpresa(e); em.persist(c2);
19             Categoria c3 = new Categoria(); c3.setNome("Sobremesas"); c3.setOrdem(3); c3.setEmpresa(e); em.persist(c3);
20             Produto p1 = new Produto(); p1.setCodigo("PRATO-001"); p1.setNome("File a Parmegiana"); p1.setPreco(new BigDecimal("45.90")); p1.setCategoria(c1); p1.setEmpresa(e);
21             em.persist(p1);
22             Produto p2 = new Produto(); p2.setCodigo("PRATO-002"); p2.setNome("Espaguete ao Molho"); p2.setPreco(new BigDecimal("32.90")); p2.setCategoria(c1); p2.setEmpresa(e);
23             em.persist(p2);
24             Produto p3 = new Produto(); p3.setCodigo("BEB-001"); p3.setNome("Suco Natural"); p3.setPreco(new BigDecimal("8.90")); p3.setCategoria(c2); p3.setEmpresa(e);
25             em.persist(p3);
26             Produto p4 = new Produto(); p4.setCodigo("BEB-002"); p4.setNome("Refrigerante Lata"); p4.setPreco(new BigDecimal("6.50")); p4.setCategoria(c2); p4.setEmpresa(e);
27             em.persist(p4);
28             Produto p5 = new Produto(); p5.setCodigo("S0B-001"); p5.setNome("Petit Gateau"); p5.setPreco(new BigDecimal("22.90")); p5.setCategoria(c3); p5.setEmpresa(e);
29             em.persist(p5);
30             for (int i = 1; i <= 10; i++) { Mesa m = new Mesa(); m.setNumero(i); m.setCapacidade(i <= 4 ? 2 : i <= 7 ? 4 : 6); m.setStatus(Mesa.Status.LIVRE); m.setEmpresa(e);
31             em.persist(m); }
32             Usuario a = new Usuario(); a.setNome("Administrador"); a.setUsername("admin"); a.setSenha(PasswordHash.hash("admin123")); a.setPapel(Usuario.Papel.ADMIN); a.setEmpresa(e);
33             em.persist(a);
34             Usuario g = new Usuario(); g.setNome("Garcom Joao"); g.setUsername("garcom"); g.setSenha(PasswordHash.hash("garcom123")); g.setPapel(Usuario.Papel.GARCOM);
35             g.setEmpresa(e); em.persist(g);
36             em.getTransaction().commit(); logger.info("Seed completed");
37             } catch (Exception ex) { if (em.getTransaction().isActive()) em.getTransaction().rollback(); logger.error("Seed error: {}", ex.getMessage(), ex); throw new
38             RuntimeException("Seed failed", ex); }
39             finally { em.close(); }
40         }
41     }

```

```

1 package com.corumbasistemas.corubafood.util;
2
3 import com.corumbasistemas.corubafood.model.*;
4 import com.corumbasistemas.corubafood.security.PasswordHash;
5 import jakarta.persistence.EntityManager;
6 import org.slf4j.*;
7
8 import java.math.BigDecimal;
9 import java.math.RoundingMode;
10 import java.util.*;
11
12 /**
13  * Seed massivo otimizado: 100 empresas, 10 usuários cada, 100 produtos cada, 10 mesas cada
14  * Usa batch processing para melhor performance.
15  */
16 public class SeedMassivo {
17     private static final Logger logger = LoggerFactory.getLogger(SeedMassivo.class);
18     private static final int TOTAL_EMPRESAS = 100;
19     private static final int USUARIOS_POR_EMPRESA = 10;
20     private static final int PRODUTOS_POR_EMPRESA = 100;
21     private static final int MESAS_POR_EMPRESA = 10;
22     private static final int CATEGORIAS_POR_EMPRESA = 5;
23     private static final int BATCH_SIZE = 10; // empresas por transação
24
25     private static final String[] NOMES_EMPRESA_PREFIX = {
26         "Restaurante", "Pizzaria", "Hamburgueria", "Café", "Sorveteria",
27         "Padaria", "Churrascaria", "Sushi Bar", "Açaiteria", "Tapiocaria"
28     };
29     private static final String[] NOMES_EMPRESA_SUFFIX = {
30         "Bom Sabor", "Sabor da Casa", "Gourmet", "do Chef", "Express",
31         "Premium", "Artesanal", "da Praça", "Central", "Prime"
32     };
33     private static final String[] NOMES_USUARIOS = {
34         "Admin", "Gerente", "Garcom", "Cozinheira", "Caixa",
35         "Atendente", "Supervisor", "Balcao", "Barman", "Pizzaiolo"
36     };
37     private static final String[] CATEGORIAS_NOME = {
38         "Pratos Principais", "Bebidas", "Sobremesas", "Petiscos", "Combos"
39     };
40     private static final String[] PRODUTOS_NOME = {
41         "File a Parmegiana", "Espagete ao Molho", "Frango Grelhado", "Picanha na Chapa",
42         "Salada Caesar", "Sopa de Legumes", "Peixe Frito", "Costela no Bafo",
43         "Risoto de Camarao", "Lasanha Bolonhesa", "Pizza Margherita", "Pizza Calabresa",
44         "Hamburguer Artesanal", "X-Burger", "X-Tudo", "X-Salada",
45         "Suco Natural", "Refrigerante", "Agua Mineral", "Cerveja Artesanal",
46         "Cafe Expresso", "Chocolate Quente", "Vitamina de Fruta", "Cha Gelado",
47         "Petit Gateau", "Pudim de Leite", "Mousse de Maracuja", "Torta de Limao",
48         "Sorvete de Creme", "Acai na Tigela", "Brownie", "Cheesecake",
49         "Batata Frita", "Calabresa Acebola", "Frango a Passarinho", "Bolinha de Queijo",
50         "Pastel de Carne", "Coxinha de Frango", "Kibe", "Espetinho",
51         "Combo Familia", "Combo Casal", "Combo Individual", "Combo Executivo",
52         "Prato do Dia", "Menu Kids", "Rodizio", "Buffet Livre"
53     };
54     private static final String[] CIDADES = {
55         "Corumba", "Campo Grande", "Dourados", "Tres Lagoas", "Ponta Pora",
56         "Navirai", "Nova Andradina", "Aquidauana", "Miranda", "Coxim"
57     };
58     private static final String[] ESTADOS = {"MS", "MT", "SP", "GO", "MG"};
59     private static final String[] ENDEREÇOS = {
60         "Rua Principal", "Av. Central", "Rua das Flores", "Av. dos Bandeirantes",
61         "Rua do Comercio", "Av. Brasil", "Rua Sao Joao", "Av. Presidente Vargas"
62     };
63     private static final Usuario.Papel[] PAPEIS = Usuario.Papel.values();
64     private static final Random rnd = new Random(42);
65
66     public static void popular() {
67         long inicio = System.currentTimeMillis();
68
69         // Verificar quantas empresas já existem
70         EntityManager emCheck = JPAUtil.getEM();
71         Long existentes = emCheck.createQuery("SELECT COUNT(e) FROM Empresa e", Long.class).getSingleResult();
72         List<String> cnpsExistentes = emCheck.createQuery("SELECT e.documento FROM Empresa e", String.class).getResultList();
73         emCheck.close();
74
75         if (existentes >= TOTAL_EMPRESAS) {
76             logger.info("DB já possui {} empresas. Seed massivo pulado.", existentes);
77             return;
78         }
79
80         logger.info("=== SEED MASSIVO: {} empresas (existem: {}) ===", TOTAL_EMPRESAS, existentes);
81
82         int criadas = 0;
83         int erros = 0;
84         Set<String> cnpsSet = new HashSet<>(cnpsExistentes);
85
86         // Processar em batches
87         for (int batchStart = 1; batchStart <= TOTAL_EMPRESAS; batchStart += BATCH_SIZE) {
88             int batchEnd = Math.min(batchStart + BATCH_SIZE - 1, TOTAL_EMPRESAS);
89             EntityManager em = JPAUtil.getEM();
90
91             try {
92                 em.getTransaction().begin();
93
94                 for (int i = batchStart; i <= batchEnd; i++) {
95                     String cnpj = gerarCNPJ(i);
96                     if (cnpsSet.contains(cnpj)) continue;
97
98                     Empresa empresa = new Empresa();
99                     empresa.setNome(NOMES_EMPRESA_PREFIX[rnd.nextInt(NOMES_EMPRESA_PREFIX.length)] + " " +
100                                 NOMES_EMPRESA_SUFFIX[rnd.nextInt(NOMES_EMPRESA_SUFFIX.length)] + " " + i);
101                     empresa.setDocumento(cnpj);
102                     empresa.setTelefone(String.format("(67) 9%04d-%04d", rnd.nextInt(10000), rnd.nextInt(10000)));
103                     empresa.setEndereco(ENDEREÇOS[rnd.nextInt(ENDEREÇOS.length)] + " " + (rnd.nextInt(500) + 1));
104                     empresa.setCidade(CIDADES[rnd.nextInt(CIDADES.length)]);
105                     empresa.setEstado(ESTADOS[rnd.nextInt(ESTADOS.length)]);
106                     empresa.setAtivo(true);
107                     em.persist(empresa);
108                     em.flush();
109                     em.clear(); // Liberar memória
110
111                     cnpsSet.add(cnpj);
112                     criadas++;
113                 }
114
115                 em.getTransaction().commit();
116                 logger.info(" → Batch {}/{ } concluído ({} empresas)", batchEnd, TOTAL_EMPRESAS, criadas);
117
118             } catch (Exception ex) {
119                 erros++;
120                 if (em.getTransaction().isActive()) em.getTransaction().rollback();
121                 logger.warn("Erro no batch {}-{:}: {}", batchStart, batchEnd, ex.getMessage());
122             } finally {
123                 em.close();
124             }
125         }
126
127         // Agora criar dados filhos (categorias, produtos, mesas, usuários) para cada empresa
128         logger.info("=== Criando dados filhos (categorias, produtos, mesas, usuários) ===");
129
130         EntityManager emList = JPAUtil.getEM();
131         List<Long> empresaIds = emList.createQuery("SELECT e.id FROM Empresa e ORDER BY e.id", Long.class).getResultList();
132         emList.close();
133
134         int empresasComDados = 0;

```

```

135
136 for (Long empresaId : empresaIds) {
137     EntityManager em = JPAUtil.getEM();
138     try {
139         // Verificar se já tem dados
140         Long countProds = em.createQuery("SELECT COUNT(p) FROM Produto p WHERE p.empresa.id=:eid", Long.class)
141             .setParameter("eid", empresaId).getSingleResult();
142         if (countProds > 0) {
143             em.close();
144             continue;
145         }
146
147         em.getTransaction().begin();
148         Empresa empresa = em.find(Empresa.class, empresaId);
149
150         // Categorias
151         Categoria[] categorias = new Categoria[CATEGORIAS_POR_EMPRESA];
152         for (int c = 0; c < CATEGORIAS_POR_EMPRESA; c++) {
153             Categoria cat = new Categoria();
154             cat.setNome(CATEGORIAS_NOME[c]);
155             cat.setDescricao(CATEGORIAS_NOME[c]);
156             cat.setOrdem(c + 1);
157             cat.setEmpresa(empresa);
158             cat.setAtivo(true);
159             em.persist(cat);
160             categorias[c] = cat;
161         }
162         em.flush();
163
164         // Produtos
165         for (int p = 0; p < PRODUTOS_POR_EMPRESA; p++) {
166             Produto prod = new Produto();
167             prod.setCodigo(String.format("%03d-%03d", empresaId, p + 1));
168             prod.setNome(PRODUTOS_NOME[rnd.nextInt(PRODUTOS_NOME.length)]);
169             prod.setPreco(BigDecimal.valueOf(rnd.nextDouble() * 80 + 5).setScale(2, RoundingMode.HALF_UP));
170             prod.setCategoria(categorias[rnd.nextInt(categorias.length)]);
171             prod.setEmpresa(empresa);
172             prod.setDisponivel(rnd.nextDouble() > 0.05);
173             prod.setAtivo(true);
174             em.persist(prod);
175         }
176
177         // Mesas
178         for (int m = 1; m <= MESAS_POR_EMPRESA; m++) {
179             Mesa mesa = new Mesa();
180             mesa.setNumero(m);
181             mesa.setCapacidade(rnd.nextInt(3) == 0 ? 6 : rnd.nextInt(2) == 0 ? 4 : 2);
182             mesa.setStatus(Mesa.Status.LIVRE);
183             mesa.setEmpresa(empresa);
184             em.persist(mesa);
185         }
186
187         // Usuários
188         for (int u = 0; u < USUARIOS_POR_EMPRESA; u++) {
189             Usuario user = new Usuario();
190             user.setNome(NOMES_USUARIOS[u]);
191             user.setUsername(NOMES_USUARIOS[u].toLowerCase() + empresaId);
192             user.setSenha(PasswordHash.hash("senha" + empresaId));
193             user.setPapel(PAPEIS[rnd.nextInt(PAPEIS.length)]);
194             user.setEmpresa(empresa);
195             user.setAtivo(true);
196             em.persist(user);
197         }
198
199         em.getTransaction().commit();
200         empresasComDados++;
201
202         if (empresasComDados % 20 == 0) {
203             logger.info(" → {}/{} empresas com dados filhos...", empresasComDados, empresaIds.size());
204         }
205
206     } catch (Exception ex) {
207         if (em.getTransaction().isActive()) em.getTransaction().rollback();
208         logger.warn("Erro dados filhos empresa {}: {}", empresaId, ex.getMessage());
209     } finally {
210         em.close();
211     }
212 }
213
214 long fim = System.currentTimeMillis();
215
216 // Contagem final
217 EntityManager emCount = JPAUtil.getEM();
218 Long totalEmpresas = emCount.createQuery("SELECT COUNT(e) FROM Empresa e", Long.class).getSingleResult();
219 Long totalUsuarios = emCount.createQuery("SELECT COUNT(u) FROM Usuario u", Long.class).getSingleResult();
220 Long totalProdutos = emCount.createQuery("SELECT COUNT(p) FROM Produto p", Long.class).getSingleResult();
221 Long totalMesas = emCount.createQuery("SELECT COUNT(m) FROM Mesa m", Long.class).getSingleResult();
222 Long totalCategorias = emCount.createQuery("SELECT COUNT(c) FROM Categoria c", Long.class).getSingleResult();
223 emCount.close();
224
225 logger.info("=== SEED MASSIVO CONCLUÍDO em {}ms ===", (fim - inicio));
226 logger.info("Empresas: {} | Usuários: {} | Produtos: {} | Mesas: {} | Categorias: {} | Erros: {}",
227     totalEmpresas, totalUsuarios, totalProdutos, totalMesas, totalCategorias, erros);
228
229
230 private static String gerarCNPJ(int seq) {
231     int base = 10000000 + seq;
232     return String.format("%02d.%03d.%03d/0001-%02d",
233         (base / 1000000) % 100,
234         (base / 1000) % 1000,
235         base % 1000,
236         seq % 100);
237 }
238 }
239

```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <persistence xmlns="https://jakarta.ee/xml/ns/persistence"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence https://jakarta.ee/xml/ns/persistence/persistence_3_0.xsd"
5     version="3.0">
6     <persistence-unit name="coruba-food-pu" transaction-type="RESOURCE_LOCAL">
7         <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
8         <class>com.corumbasistemas.corubafood.model.Empresa</class>
9         <class>com.corumbasistemas.corubafood.model.Usuario</class>
10        <class>com.corumbasistemas.corubafood.model.Categoria</class>
11        <class>com.corumbasistemas.corubafood.model.Produto</class>
12        <class>com.corumbasistemas.corubafood.model.Mesa</class>
13        <class>com.corumbasistemas.corubafood.model.Pedido</class>
14        <class>com.corumbasistemas.corubafood.model.ItemPedido</class>
15        <class>com.corumbasistemas.corubafood.model.Pagamento</class>
16        <class>com.corumbasistemas.corubafood.model.PedidoDelivery</class>
17        <class>com.corumbasistemas.corubafood.model.ItemPedidoDelivery</class>
18        <properties>
19            <property name="jakarta.persistence.jdbc.driver" value="org.sqlite.JDBC"/>
20            <property name="jakarta.persistence.jdbc.url" value="jdbc:sqlite:coruba-food.db"/>
21            <property name="hibernate.dialect" value="org.hibernate.community.dialect.SQLiteDialect"/>
22            <property name="hibernate.hbm2ddl.auto" value="update"/>
23            <property name="hibernate.show_sql" value="false"/>
24            <property name="hibernate.format_sql" value="false"/>
25            <property name="hibernate.c3p0.min_size" value="2"/>
26            <property name="hibernate.c3p0.max_size" value="10"/>
27            <property name="hibernate.c3p0.timeout" value="300"/>
28            <property name="hibernate.c3p0.max_statements" value="50"/>
29        </properties>
30    </persistence-unit>
31 </persistence>
32
```



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3   <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
4     <encoder>
5       <pattern>%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
6     </encoder>
7   </appender>
8
9   <!-- Desativar logs verbose do Hibernate -->
10  <logger name="org.hibernate" level="WARN"/>
11  <logger name="org.hibernate.orm.results" level="OFF"/>
12  <logger name="org.hibernate.sql.results" level="OFF"/>
13  <logger name="org.hibernate.resource.jdbc" level="OFF"/>
14  <logger name="org.hibernate.engine.jdbc" level="OFF"/>
15  <logger name="org.hibernate.cache" level="OFF"/>
16  <logger name="org.hibernate.stat" level="OFF"/>
17  <logger name="org.hibernate.hql" level="OFF"/>
18  <logger name="org.hibernate.prepared" level="OFF"/>
19
20  <!-- Manter INFO da aplicacao -->
21  <logger name="com.corumbasistemas" level="INFO"/>
22
23  <root level="INFO">
24    <appender-ref ref="CONSOLE"/>
25  </root>
26 </configuration>
27
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <?import javafx.geometry.*?>
4 <?import javafx.scene.control.*?>
5 <?import javafx.scene.layout.*?>
6 <?import javafx.scene.text.*?>
7
8 <BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
9     fx:controller="com.corumbasistemas.corubafood.controller.ConfigIntegracaoController"
10     style="-fx-background-color: #0f0f23;"
11
12 <top>
13     <HBox spacing="12" alignment="CENTER LEFT"
14         style="-fx-background-color: #16213e; -fx-padding: 12 20;">
15         <Label text="o Configuração de APIs"
16             style="-fx-text-fill: #4ecca3; -fx-font-size: 20; -fx-font-weight: bold;"/>
17         <Region HBox.hgrow="ALWAYS"/>
18         <Button text=" Voltar" onAction="#handleVoltar"
19             style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 16;"/>
20     </HBox>
21 </top>
22
23 <center>
24     <ScrollPane fitToWidth="true" style="-fx-background: #0f0f23;">
25         <VBox spacing="16" style="-fx-padding: 20;">
26
27             <!-- ===== iFood ===== -->
28             <VBox spacing="10" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
29                 <HBox spacing="8" alignment="CENTER LEFT">
30                     <Label text="🍷 iFood" style="-fx-text-fill: #eald2c; -fx-font-size: 18; -fx-font-weight: bold;"/>
31                     <Label fx:id="lblIfoodStatus" text="🍷 Não configurado" style="-fx-text-fill: #888;"/>
32                 </HBox>
33                 <Label text="Credenciais do portal developer.ifood.com.br"
34                     style="-fx-text-fill: #888; -fx-font-size: 11;"/>
35
36                 <GridPane vgap="8" hgap="12">
37                     <columnConstraints>
38                         <ColumnConstraints minWidth="130" prefWidth="150"/>
39                         <ColumnConstraints hgrow="ALWAYS"/>
40                     </columnConstraints>
41
42                     <Label text="Client ID:" GridPane.rowIndex="0" GridPane.columnIndex="0"
43                         style="-fx-text-fill: #aaa;"/>
44                     <TextField fx:id="txtIfoodClientId" GridPane.rowIndex="0" GridPane.columnIndex="1"
45                         promptText="Ex: abc123def456..."
46                         style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
47
48                     <Label text="Client Secret:" GridPane.rowIndex="1" GridPane.columnIndex="0"
49                         style="-fx-text-fill: #aaa;"/>
50                     <PasswordField fx:id="txtIfoodClientSecret" GridPane.rowIndex="1" GridPane.columnIndex="1"
51                         promptText="Ex: xyz789secret..."
52                         style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
53
54                     <Label text="Merchant ID:" GridPane.rowIndex="2" GridPane.columnIndex="0"
55                         style="-fx-text-fill: #aaa;"/>
56                     <TextField fx:id="txtIfoodMerchantId" GridPane.rowIndex="2" GridPane.columnIndex="1"
57                         promptText="Ex: 12345"
58                         style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
59                 </GridPane>
60
61                 <HBox spacing="8">
62                     <Button text="🔑 Salvar" onAction="#handleIfoodSalvar"
63                         style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 20;"/>
64                     <Button text="🧪 Testar Conexão" onAction="#handleIfoodTestar"
65                         style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8 20;"/>
66                 </HBox>
67             </VBox>
68
69             <!-- ===== 99Food ===== -->
70             <VBox spacing="10" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
71                 <HBox spacing="8" alignment="CENTER LEFT">
72                     <Label text="🍷 99Food" style="-fx-text-fill: #fffd700; -fx-font-size: 18; -fx-font-weight: bold;"/>
73                     <Label fx:id="lbl99Status" text="🍷 Não configurado" style="-fx-text-fill: #888;"/>
74                 </HBox>
75                 <Label text="Credenciais do painel do parceiro 99Food"
76                     style="-fx-text-fill: #888; -fx-font-size: 11;"/>
77
78                 <GridPane vgap="8" hgap="12">
79                     <columnConstraints>
80                         <ColumnConstraints minWidth="130" prefWidth="150"/>
81                         <ColumnConstraints hgrow="ALWAYS"/>
82                     </columnConstraints>
83
84                     <Label text="Client ID:" GridPane.rowIndex="0" GridPane.columnIndex="0"
85                         style="-fx-text-fill: #aaa;"/>
86                     <TextField fx:id="txt99ClientId" GridPane.rowIndex="0" GridPane.columnIndex="1"
87                         promptText="Ex: 99abc123..."
88                         style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
89
90                     <Label text="Client Secret:" GridPane.rowIndex="1" GridPane.columnIndex="0"
91                         style="-fx-text-fill: #aaa;"/>
92                     <PasswordField fx:id="txt99ClientSecret" GridPane.rowIndex="1" GridPane.columnIndex="1"
93                         promptText="Ex: 99xyz789secret..."
94                         style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
95
96                     <Label text="Merchant ID:" GridPane.rowIndex="2" GridPane.columnIndex="0"
97                         style="-fx-text-fill: #aaa;"/>
98                     <TextField fx:id="txt99MerchantId" GridPane.rowIndex="2" GridPane.columnIndex="1"
99                         promptText="Ex: 67890"
100                         style="-fx-background-color: #0f0f23; -fx-text-fill: white; -fx-prompt-text-fill: #555;"/>
101                 </GridPane>
102
103                 <HBox spacing="8">
104                     <Button text="🔑 Salvar" onAction="#handle99Salvar"
105                         style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 20;"/>
106                     <Button text="🧪 Testar Conexão" onAction="#handle99Testar"
107                         style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8 20;"/>
108                 </HBox>
109             </VBox>
110
111             <!-- ===== Configurações Gerais ===== -->
112             <VBox spacing="10" style="-fx-background-color: #16213e; -fx-padding: 16; -fx-background-radius: 10;">
113                 <Label text="🔄 Sincronização" style="-fx-text-fill: #4ecca3; -fx-font-size: 16; -fx-font-weight: bold;"/>
114
115                 <GridPane vgap="8" hgap="12">
116                     <columnConstraints>
117                         <ColumnConstraints minWidth="180" prefWidth="200"/>
118                         <ColumnConstraints hgrow="ALWAYS"/>
119                     </columnConstraints>
120
121                     <Label text="Intervalo de busca:" GridPane.rowIndex="0" GridPane.columnIndex="0"
122                         style="-fx-text-fill: #aaa;"/>
123                     <HBox spacing="6" alignment="CENTER LEFT" GridPane.rowIndex="0" GridPane.columnIndex="1">
124                         <Spinner fx:id="spnPollInterval" prefWidth="100"
125                             style="-fx-background-color: #0f0f23;"/>
126                         <Label text="segundos" style="-fx-text-fill: #888;"/>
127                     </HBox>
128
129                     <Label text="Iniciar automaticamente:" GridPane.rowIndex="1" GridPane.columnIndex="0"
130                         style="-fx-text-fill: #aaa;"/>
131                     <CheckBox fx:id="chkAutoStart" GridPane.rowIndex="1" GridPane.columnIndex="1"
132                         style="-fx-text-fill: white;"/>
133
134                     <Label text="Alerta sonoro:" GridPane.rowIndex="2" GridPane.columnIndex="0"

```

```

135         style="-fx-text-fill: #aaa;"/>
136         <CheckBox fx:id="chkSoundAlert" GridPane.rowIndex="2" GridPane.columnIndex="1"
137             style="-fx-text-fill: white;"/>
138     </GridPane>
139
140     <Button text="⚙ Salvar Configurações" onAction="#handleSalvarGeral"
141         style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 8 20;"/>
142 </VBox>
143
144 <!-- ===== Instruções ===== -->
145 <VBox spacing="8" style="-fx-background-color: #1a1a2e; -fx-padding: 16; -fx-background-radius: 10;">
146     <Label text="📖 Como obter as credenciais" style="-fx-text-fill: #4ecca3; -fx-font-size: 14; -fx-font-weight: bold;"/>
147     <Label text="iFood: Acesse developer.ifood.com.br → Crie um app → Copie Client ID, Client Secret e Merchant ID"
148         wrapText="true" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
149     <Label text="99Food: Entre em contato com o suporte 99Food → Solicite acesso API → Receba as credenciais"
150         wrapText="true" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
151     <Label text="⚠ Suas credenciais ficam salvas APENAS no computador do restaurante. Não são enviadas para nenhum servidor."
152         wrapText="true" style="-fx-text-fill: #f77f00; -fx-font-size: 12; -fx-font-weight: bold;"/>
153 </VBox>
154
155 </VBox>
156 </ScrollPane>
157 </center>
158
159 <bottom>
160     <HBox style="-fx-background-color: #16213e; -fx-padding: 6 15;">
161         <Label fx:id="lblStatusBar" text="Carregando..." style="-fx-text-fill: #4ecca3; -fx-font-size: 11;"/>
162     </HBox>
163 </bottom>
164 </BorderPane>
165

```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <?import javafx.geometry.*?>
4 <?import javafx.scene.control.*?>
5 <?import javafx.scene.layout.*?>
6 <?import javafx.scene.text.*?>
7
8 <BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
9   fx:controller="com.corumbasistemas.corubafood.controller.DeliveryController"
10   style="-fx-background-color: #0f0f23;"
11
12   <!-- Top: Header com título e info do usuário -->
13   <top>
14     <VBox spacing="5" style="-fx-background-color: #16213e; -fx-padding: 12 20 12 20;">
15       <HBox alignment="CENTER_LEFT" spacing="15">
16         <Label text="🍴 ConnectFood" style="-fx-text-fill: #4ecca3; -fx-font-size: 22; -fx-font-weight: bold;"/>
17         <Pane HBox.hgrow="ALWAYS"/>
18         <Button text="⚙ Configurar APIs" onAction="#handleConfigurar"
19           style="-fx-background-color: #16213e; -fx-text-fill: #4ecca3; -fx-font-weight: bold; -fx-padding: 8 14;"/>
20         <Button text="↩ Voltar" onAction="#handleVoltar"
21           style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 8 16;"/>
22         <Label fx:id="lblUsuario" text="Usuário" style="-fx-text-fill: #888; -fx-font-size: 12;"/>
23       </HBox>
24
25       <!-- Abas de filtro -->
26       <HBox spacing="8" alignment="CENTER_LEFT">
27         <Label text="Filtrar:" style="-fx-text-fill: #aaa; -fx-font-size: 12;"/>
28         <ToggleButton fx:id="btnAbaTodos" text="🗖 Todos" selected="true"
29           style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;"/>
30         <ToggleButton fx:id="btnAbaIfood" text="🍷 iFood"
31           style="-fx-background-color: #eald2c; -fx-text-fill: white; -fx-font-weight: bold;"/>
32         <ToggleButton fx:id="btnAba99food" text="🍷 99Food"
33           style="-fx-background-color: #fffd700; -fx-text-fill: #333; -fx-font-weight: bold;"/>
34         <Pane HBox.hgrow="ALWAYS"/>
35         <Button text="📄 Simular Pedido" onAction="#handleSimularPedido"
36           style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 6 14;"/>
37       </HBox>
38     </VBox>
39   </top>
40
41   <!-- Center: Lista de pedidos e detalhes -->
42   <center>
43     <SplitPane dividerPositions="0.4" orientation="HORIZONTAL">
44       <!-- Lado esquerdo: Lista + Dashboard -->
45       <VBox spacing="10" style="-fx-padding: 10; -fx-background-color: #0f0f23;">
46         <!-- Cards do Dashboard -->
47         <HBox spacing="8" alignment="CENTER">
48           <!-- iFood -->
49           <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8; HBox.hgrow="ALWAYS">
50             <Label text="🍷 iFood" style="-fx-text-fill: #eald2c; -fx-font-weight: bold;"/>
51             <Label fx:id="lblTotalIfood" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
52           </VBox>
53           <!-- 99Food -->
54           <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8; HBox.hgrow="ALWAYS">
55             <Label text="🍷 99Food" style="-fx-text-fill: #fffd700; -fx-font-weight: bold;"/>
56             <Label fx:id="lblTotal99food" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
57           </VBox>
58           <!-- Novos -->
59           <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8; HBox.hgrow="ALWAYS">
60             <Label text="📄 Novos" style="-fx-text-fill: #4ecca3; -fx-font-weight: bold;"/>
61             <Label fx:id="lblNovos" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
62           </VBox>
63           <!-- Preparando -->
64           <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8; HBox.hgrow="ALWAYS">
65             <Label text="📄 Preparando" style="-fx-text-fill: #f77f00; -fx-font-weight: bold;"/>
66             <Label fx:id="lblPreparando" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
67           </VBox>
68           <!-- Prontos -->
69           <VBox alignment="CENTER" style="-fx-background-color: #16213e; -fx-padding: 10; -fx-background-radius: 8; HBox.hgrow="ALWAYS">
70             <Label text="📄 Prontos" style="-fx-text-fill: #84cc16; -fx-font-weight: bold;"/>
71             <Label fx:id="lblProntos" text="0" style="-fx-text-fill: white; -fx-font-size: 24; -fx-font-weight: bold;"/>
72           </VBox>
73         </HBox>
74
75         <!-- Título da lista -->
76         <Label text="🗖 Pedidos Recebidos" style="-fx-text-fill: #4ecca3; -fx-font-size: 16; -fx-font-weight: bold;"/>
77
78         <!-- Lista de pedidos -->
79         <ListView fx:id="lvPedidos" VBox.vgrow="ALWAYS"
80           style="-fx-background-color: #16213e; -fx-control-inner-background: #16213e; -fx-text-fill: white;"/>
81         <placeholder>
82           <Label text="Nenhum pedido de delivery" style="-fx-text-fill: #666;"/>
83         </placeholder>
84       </ListView>
85     </VBox>
86
87     <!-- Lado direito: Detalhes + Ações -->
88     <VBox spacing="10" style="-fx-padding: 10; -fx-background-color: #0f0f23;">
89       <Label fx:id="lblDetalhesTitulo" text="Selecione um pedido"
90         style="-fx-text-fill: #4ecca3; -fx-font-size: 16; -fx-font-weight: bold;"/>
91
92       <TextArea fx:id="txtDetalhesConteudo" editable="false" wrapText="true"
93         VBox.vgrow="ALWAYS" prefRowCount="20"
94         style="-fx-control-inner-background: #16213e; -fx-text-fill: #ddd; -fx-font-family: 'Consolas', monospace; -fx-font-size: 13;"/>
95
96       <!-- Botões de ação -->
97       <HBox spacing="8" alignment="CENTER">
98         <Button fx:id="btnAvancar" text="▶ Avançar Status" onAction="#handleAvancarStatus"
99           style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold; -fx-padding: 10 20; -fx-font-size: 14;"/>
100         <Button fx:id="btnCancelar" text="❌ Cancelar" onAction="#handleCancelarPedido"
101           style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-weight: bold; -fx-padding: 10 20; -fx-font-size: 14;"/>
102       </HBox>
103     </VBox>
104   </SplitPane>
105 </center>
106
107   <!-- Bottom: Barra de status -->
108   <bottom>
109     <HBox style="-fx-background-color: #16213e; -fx-padding: 6 15 6 15;">
110       <Label fx:id="lblStatusBar" text="Carregando..." style="-fx-text-fill: #4ecca3; -fx-font-size: 11;"/>
111     </HBox>
112   </bottom>
113 </BorderPane>
114

```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <?import javafx.scene.control.*?>
3 <?import javafx.scene.layout.*?>
4 <?import javafx.scene.text.*?>
5 <?import javafx.geometry.*?>
6
7 <AnchorPane prefWidth="420" prefHeight="520" style="-fx-background-color: #1a1a2e;"
8             xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
9             fx:controller="com.corumbasistemas.corubafood.controller.LoginController">
10     <VBox alignment="CENTER" spacing="16" AnchorPane.leftAnchor="30" AnchorPane.rightAnchor="30" AnchorPane.topAnchor="30">
11         <Label text="M Corumbá Food" style="-fx-font-size: 28px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
12         <Label text="MesaPronta v2.0" style="-fx-font-size: 14px; -fx-text-fill: #a0a0b0;"/>
13
14         <VBox spacing="4">
15             <Label text="CNPJ/CPF da Empresa" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
16             <TextField fx:id="txtDocumento" promptText="00.000.000/0000-00" style="-fx-font-size: 14px;"/>
17         </VBox>
18
19         <VBox spacing="4">
20             <Label text="Usuário" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
21             <TextField fx:id="txtUsername" promptText="admin" style="-fx-font-size: 14px;"/>
22         </VBox>
23
24         <VBox spacing="4">
25             <Label text="Senha" style="-fx-text-fill: #c0c0d0; -fx-font-size: 12px;"/>
26             <PasswordField fx:id="txtSenha" promptText="*****" style="-fx-font-size: 14px;"/>
27         </VBox>
28
29         <Label fx:id="lblMensagem" style="-fx-font-size: 12px;"/>
30
31         <Button fx:id="btnEntrar" text="ENTRAR" onAction="#handleEntrar"
32               prefWidth="360" prefHeight="44"
33               style="-fx-background-color: #e94560; -fx-text-fill: white; -fx-font-size: 16px; -fx-font-weight: bold; -fx-cursor: hand;"/>
34
35         <ProgressIndicator fx:id="progress" visible="false"/>
36     </VBox>
37 </AnchorPane>
```

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <?import javafx.scene.control.*?>
3 <?import javafx.scene.layout.*?>
4 <?import javafx.scene.text.*?>
5 <?import javafx.geometry.*?>
6
7 <BorderPane prefWidth="1200" prefHeight="800" style="-fx-background-color: #0f0f23;"
8     xmlns="http://javafx.com/javafx/21" xmlns:fx="http://javafx.com/fxml/1"
9     fx:controller="com.corumbasistemas.corubafood.controller.PrincipalController">
10     <!-- TOP: Header -->
11     <top>
12         <HBox spacing="12" alignment="CENTER LEFT" style="-fx-background-color: #16213e; -fx-padding: 10 20;">
13             <Label text="M MesaPronta" style="-fx-font-size: 20px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
14             <Region HBox.hgrow="ALWAYS"/>
15             <Label fx:id="lblUsuario" text="Usuário" style="-fx-text-fill: #a0a0b0; -fx-font-size: 13px;"/>
16             <Button text="🚚 Delivery" onAction="#handleDelivery" style="-fx-background-color: #eald2c; -fx-text-fill: white; -fx-font-weight: bold;"/>
17             <Button text="Sair" onAction="#handleSair" style="-fx-background-color: #e94560; -fx-text-fill: white;"/>
18         </HBox>
19     </top>
20
21     <!-- CENTER: Mesas -->
22     <center>
23         <ScrollPane fitToWidth="true" style="-fx-background: #0f0f23;">
24             <FlowPane fx:id="fpMesas" hgap="12" vgap="12" style="-fx-padding: 16;"/>
25         </ScrollPane>
26     </center>
27
28     <!-- RIGHT: Pedido da Mesa -->
29     <right>
30         <VBox spacing="8" prefWidth="380" style="-fx-background-color: #1a1a2e; -fx-padding: 12;">
31             <Label fx:id="lblMesaTitulo" text="Selecione uma mesa" style="-fx-font-size: 18px; -fx-font-weight: bold; -fx-text-fill: #e94560;"/>
32
33             <HBox spacing="8">
34                 <Label text="Status:" style="-fx-text-fill: #a0a0b0;"/>
35                 <Label fx:id="lblMesaStatus" text="-" style="-fx-text-fill: #4ecca3; -fx-font-weight: bold;"/>
36             </HBox>
37
38             <Separator style="-fx-background-color: #333;"/>
39
40             <Label text="Itens do Pedido" style="-fx-font-size: 14px; -fx-text-fill: #c0c0d0;"/>
41             <ListView fx:id="lvItens" VBox.vgrow="ALWAYS" style="-fx-control-inner-background: #0f0f23;"/>
42
43             <HBox spacing="8" alignment="CENTER LEFT">
44                 <Label text="Total:" style="-fx-text-fill: #a0a0b0; -fx-font-size: 16px;"/>
45                 <Label fx:id="lblTotal" text="R$ 0,00" style="-fx-font-size: 22px; -fx-font-weight: bold; -fx-text-fill: #4ecca3;"/>
46             </HBox>
47
48             <HBox spacing="8">
49                 <Button text="+ Adicionar Item" onAction="#handleAdicionarItem" prefWidth="170"
50                     style="-fx-background-color: #4ecca3; -fx-text-fill: #0f0f23; -fx-font-weight: bold;"/>
51                 <Button text="🗑 Remover" onAction="#handleRemoverItem" prefWidth="170"
52                     style="-fx-background-color: #e94560; -fx-text-fill: white;"/>
53             </HBox>
54
55             <HBox spacing="8">
56                 <Button text="✓ Fechar Pedido" onAction="#handleFecharPedido" prefWidth="170"
57                     style="-fx-background-color: #00b4d8; -fx-text-fill: white; -fx-font-weight: bold;"/>
58                 <Button text="💰 Pagamento" onAction="#handlePagamento" prefWidth="170"
59                     style="-fx-background-color: #f77f00; -fx-text-fill: white; -fx-font-weight: bold;"/>
60             </HBox>
61         </VBox>
62     </right>
63
64     <!-- BOTTOM: Status bar -->
65     <bottom>
66         <HBox spacing="8" style="-fx-background-color: #16213e; -fx-padding: 6 20;">
67             <Label fx:id="lblStatusBar" text="Pronto" style="-fx-text-fill: #4ecca3; -fx-font-size: 11px;"/>
68         </HBox>
69     </bottom>
70 </BorderPane>
71

```